

OPEN
Compute Project

Hardware Fault Management Project

System Debug Specification

0.2

Modular Base Specification
Effective October 27, 2026

Author: (See Acknowledgements Section)

Current Template Version:

3 Layer (Base, Design and Product) Specification Template V1.6.0

Table of Contents

1 License **7**
2 Acknowledgements **8**
3 Revision History **9**
4 Compliance with OCP Tenets **10**
4.1 Openness 10
4.2 Efficiency 10
4.3 Impact 10
4.4 Scale 10
4.5 Sustainability 10
5 Introduction **11**
5.1 Goals 11
5.2 Document Scope 12
5.2.1 In Scope 12
5.2.2 Out of Scope 12
5.3 Audience 12
5.4 Objectives 13
5.4.1 Standardization of Reporting Debug Data 13
5.4.2 Security Controls 13
5.4.3 Vendor-Specific Extensions 13
5.5 Expected Outcomes 13
6 Terminology **14**
6.1 Syntax and Conventions 14
6.1.1 Special Terms 14
6.1.2 Number Radix 14
6.1.3 JSON Keys 14
6.2 Definitions 14
6.3 Abbreviations 15
6.4 Acronyms 15
7 Technical Overview (Informative) **17**
7.1 Product Development Life Cycle 17
7.1.1 Engineering Validation Test (EVT) 17
7.1.2 Design Validation Test (DVT) 18
7.1.3 Production Validation Testing (PVT) 18
7.1.4 Quarantined: Open Enclosure 19
7.1.5 Quarantined: Production Enclosure 19
7.1.6 Production: Accessible Rack 19
7.1.7 Production: - Remotely Managed 20
7.2 Execution Flow with Debug 20
7.2.1 Boot Crash Handling 22
7.2.2 Preemptive Issues Handling 22
7.2.3 Performance Issues Handling 23

7.2.4 Failure Handling 23
7.2.5 Crash Handling 24
7.3 High-Level Debug Architecture 25
7.4 Configuration vs Physical Error Determination Flow 25
7.5 Content Arrangement 26
8 Security and Privacy 27
9 Data Stacks 29
9.1 Code Currency 29
9.1.1 File Format and Schema 29
9.1.2 Code Currency Generation 33
9.1.3 Requirements 34
9.2 Status Dump 41
9.2.1 Data Selection Methodology 42
9.2.2 File Format and Schema 43
9.2.3 Status Dump Generation 46
9.2.4 Requirements 46
10 Future Work 56
Appendices 57
A Code Currency Examples 57
B Status Dump Examples 62
References 73

List of Figures

1 Product Development Life Cycle 17
2 Execution Flow 21
3 System Debug Architecture 25
4 Configuration vs Physical Error Determination Flow 26
5 Code Currency Relationships 30
6 Code Currency Creation 34

List of Tables

1 Code Currency Node-Level Elements Definition 31
2 Code Currency assembly Elements Definition 32
3 Code Currency component Elements Definition 32
4 Status Dump Node-Level Elements Definition 43
5 Status Dump assembly Elements Definition 44
6 Status Dump component Elements Definition 45

1 License

Open Web Foundation (OWF) CLA

Contributions to this Specification are made under the terms and conditions set forth in **Modified Open Web Foundation Agreement 0.9 (OWFa 0.9)**. (As of October 16, 2024) (“Contribution License”) by:

Aivres, AMD, Dell, Intel, NVIDIA, Qualcomm, Vertiv

Usage of this Specification is governed by the terms and conditions set forth in **Modified OWFa 0.9 Final Specification Agreement (FSA)** (As of October 16, 2024) **(“Specification License”)**.

You can review the applicable Specification License(s) referenced above by the contributors to this Specification on the OCP website at https://www.opencompute.org/contributions/templates-agreements.

For actual executed copies of either agreement, please contact OCP directly.

NOTWITHSTANDING THE FOREGOING LICENSES, THIS SPECIFICATION IS PROVIDED BY OCP “AS IS” AND OCP EXPRESSLY DISCLAIMS ANY WARRANTIES (EXPRESS, IMPLIED, OR OTHERWISE), INCLUDING IMPLIED WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR A PARTICULAR PURPOSE, OR TITLE, RELATED TO THE SPECIFICATION. NOTICE IS HEREBY GIVEN, THAT OTHER RIGHTS NOT GRANTED AS SET FORTH ABOVE, INCLUDING WITHOUT LIMITATION, RIGHTS OF THIRD PARTIES WHO DID NOT EXECUTE THE ABOVE LICENSES, MAY BE IMPLICATED BY THE IMPLEMENTATION OF OR COMPLIANCE WITH THIS SPECIFICATION. OCP IS NOT RESPONSIBLE FOR IDENTIFYING RIGHTS FOR WHICH A LICENSE MAY BE REQUIRED IN ORDER TO IMPLEMENT THIS SPECIFICATION. THE ENTIRE RISK AS TO IMPLEMENTING OR OTHERWISE USING THE SPECIFICATION IS ASSUMED BY YOU. IN NO EVENT WILL OCP BE LIABLE TO YOU FOR ANY MONETARY DAMAGES WITH RESPECT TO ANY CLAIMS RELATED TO, OR ARISING OUT OF YOUR USE OF THIS SPECIFICATION, INCLUDING BUT NOT LIMITED TO ANY LIABILITY FOR LOST PROFITS OR ANY CONSEQUENTIAL, INCIDENTAL, INDIRECT, SPECIAL OR PUNITIVE DAMAGES OF ANY CHARACTER FROM ANY CAUSES OF ACTION OF ANY KIND WITH RESPECT TO THIS SPECIFICATION, WHETHER BASED ON BREACH OF CONTRACT, TORT (INCLUDING NEGLIGENCE), OR OTHERWISE, AND EVEN IF OCP HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

2 Acknowledgements

The contributors of this Specification would like to acknowledge the following:

- Marko Bartscherer (Intel)
- Austin Bolen (Dell)
- Enrico Carrieri (Qualcomm Technologies, Inc.)
- Donald Gaither (Vertiv)
- Justin King (AMD)
- Rolf Kuehnis (Lauterbach)
- Jason Peck (NVIDIA)
- James Zou (Aivres)

3 Revision History

Rev.	Date	Guiding Contributor(s)	Description
0.1	2026-09-09	Marko B., Enrico C., Donald G., James Z.	Initial Release
0.2	2026-10-27	Enrico C., Marko B.	Added KPIs, Cleaned up Requirements and Informative Material, Fixed common terminology

4 Compliance with OCP Tenets

4.1 Openness

This specification follows the OCP tenet of openness by utilizing publicly available source materials, open development processes, and community review. The specification is self-contained and avoids dependencies on proprietary or restricted documentation. Requirements are defined within this document or reference publicly available standards, enabling transparent implementation, validation, and interoperability across vendors and deployments.

4.2 Efficiency

This specification aligns with the OCP tenet of efficiency by reducing Mean Time to Debug (MTTD) through standardized debug discovery, data collection, and diagnostic workflows. The specification prioritizes reuse of existing infrastructure, industry standards, and established debug mechanisms to improve interoperability, minimize implementation overhead, and accelerate root cause analysis.

4.3 Impact

This specification aligns with the OCP tenet of impact by standardizing debug data models and collection mechanisms across vendors. A common representation of debug information improves interoperability, enables reuse of analysis and decoding tools, and reduces reliance on vendor-specific workflows. This simplifies datacenter operations and supports efficient management of heterogeneous environments.

4.4 Scale

This specification aligns with the OCP tenet of scale by providing a hierarchical and vendor-neutral approach to debug data management that spans datacenter deployments from clusters and racks to individual nodes and components. Standardized data models and interfaces allow systems of varying size, complexity, and architectural composition to be represented consistently.

The specification is designed to accommodate diverse implementations across vendors while enabling common operational workflows. This approach supports scalable deployment, simplifies integration in heterogeneous environments, and enables consistent debug data collection and analysis across the datacenter infrastructure.

4.5 Sustainability

This specification aligns with the OCP tenet of sustainability by prioritizing reuse of existing infrastructure, interfaces, and industry standards. Leveraging established capabilities minimizes implementation impact, reduces complexity, and enables efficient adoption across existing datacenter environments.

5 Introduction

With the rise of large datacenter and AI systems, nodes and components are becoming more diverse and increasing in size and complexity. For example, training LLM models in datacenters requires orchestration of clusters consisting of a large number of server systems. The scale of such clusters is determined by factors including model size (parameters) and training dataset volume. During long-duration training, system failures are expected and can result in workload interruption, reduced efficiency, increased operational cost and delays in releasing new AI versions.

This increasing complexity requires the ability to validate, diagnose, debug, and optimize beyond the component level. Before the publication of this Specification, the datacenter community was lacking a standardized methodology to extract diagnostic data and perform debug operations across the datacenter at different levels, from components to cluster. There was a need to develop a Specification that defines how debug data is exchanged between all components across the datacenter and the debug tooling. The community desired to standardize the necessary control and transport protocols enabling uniform debug of diverse systems.

The following challenges are commonly observed in datacenter debugging environments: It is difficult to identify the failure root cause when a training job terminates unexpectedly. Normally it takes multiple iterations to isolate and resolve hardware-related issues. Additional hardware failures can occur after the hardware issue was resolved. Sometimes, it is difficult to distinguish between hardware-induced and software-induced failures.

System designs that comply with OCP Fault Management Infrastructure Requirements [1] and OCP GPU & Accelerator RAS Requirements [2] improve observability and reliability. However, such compliance does not eliminate system failures. Efficient system-level debugging remains a critical requirement for datacenter operations.

This Specification addresses debugging components and nodes within a datacenter across the lifecycle of the datacenter. It will prescribe an architecture to communicate various debug data in a common method and data models across the datacenter. The Specification will define interfaces, protocols, APIs, and data models incorporating existing work from other OCP Workstreams and other standards bodies improving the effectiveness of these for debugging in the datacenter. A system-level debugging framework is created to improve fault observability, root cause analysis (RCA), and remediation efficiency in server systems deployed in datacenters. Scalability across the lifecycle of the datacenter including but not limited to accessible ports, diagnostic and debug capabilities, and security and privacy requirements.

5.1 Goals

The main goal of this Specification is to prescribe an architecture, including interfaces, protocols, APIs, and data models, that communicates debug data across the datacenter. This will be done by:

- Defining the use of common connectivity, transports, and protocols.
  - Define a common communication “pipe” for debug data/messages.
  - Data/message payloads stay vendor-specific.
- Using existing specifications/standards when available.
  - Leverage existing standard transports (e.g., USB) and protocols (e.g., PLDM).
- Allowing flexibility while providing interoperability.

  - Flexibility in how the system is put together.
  - Debug capabilities need to be discoverable and self-describing.
  - Interoperability with both different components and different debug tooling.
  - Data needs to be opaque to components in the middle (e.g., BMC).
- Providing scalability across the lifecycle of development.
  - Use starting from manufacturing through to the at-scale, in-field deployment.

5.2 Document Scope

5.2.1 In Scope

This Specification is part of a larger OCP Hardware Management work and is designed to supplement that greater work. The Specification is focused on new capabilities that complement existing specified debug capabilities such as crash dumps. For this version, the new capabilities do not put any sensitive material at risk. Note that capabilities the handle sensitive material will be described in future versions. These capabilities are meant to be used in all stages of development including production where real tenant workloads are present. This Specification will be vendor-neutral, though provide the flexibility to include vendor-specific data alongside the standardized data.

Besides debug capabilities, this Specification will provide a common set of concepts including the product development lifecycle and a high-level debug architecture. These concepts frame the capabilities in the overall system and environment for debugging. Security of the debug capabilities in these environments will also be addressed.

5.2.2 Out of Scope

This Specification will not cover capabilities related to performance monitoring and telemetry.

In addition, this Specification will not cover existing material already covered by another OCP specification even if it is related to the goals stated in Section 5.1. This includes the following:

- Error Events - covered by the RAS API Specification [3].
- Crash Dump - covered by the RAS API Specification [3], Hyperscale CPU Debug and RAS Requirements [4], and GPU & Accelerator RAS Requirements [2] documents.
- Register Dump - covered by the Hyperscale CPU Debug and RAS Requirements [4] and GPU & Accelerator RAS Requirements [2] documents.
- Extended Register Dump - covered by the Hyperscale CPU Debug and RAS Requirements [4]
- State Dump - covered by the Hyperscale CPU Debug and RAS Requirements [4] document
- Debug Dump - covered by the Hyperscale CPU Debug and RAS Requirements [4] document
- Trace Dump - covered by the Hyperscale CPU Debug and RAS Requirements [4] document

These will be incorporated (generically) or referenced in later versions of this specification.

5.3 Audience

The primary audience of this Specification is datacenter operators and field service technicians who need to quickly debug datacenter systems and components. The secondary audience is datacenter component and system vendors who wish to design OCP compliant hardware.

5.4 Objectives

This Specification defines requirements in the following areas: standardization of reporting debug data, definition of debug levels based on system complexity, provision of security controls for debug data, and support for vendor-specific diagnostic extensions.

5.4.1 Standardization of Reporting Debug Data

It is important for a system to have a standard set of debug data report items, data formats, data collection methods, and reporting sequences. All standardized debug data is represented using structured JSON key-value pairs to ensure consistency and machine readability. The system will support access to debug data via industry-standard interfaces such as Redfish. Each reported event will include timestamp information sufficient to enable event ordering and cross-component correlation.

5.4.2 Security Controls

The system will provide mechanisms to protect sensitive material collected as debug data. Such security mechanisms include configurable access control for debug data, encoding or protection of configuration and status register contents, and integration of controls with platform-level security configuration (e.g., BIOS settings). Debug data collection, storage, and access mechanisms will comply with the configured security policies.

5.4.3 Vendor-Specific Extensions

The system will enable additional diagnostic capabilities through vendor-specific extensions. Standardized data structures will be preserved for interoperability. Vendor-specific data fields can be included as extensions to the standard schema. The system will support access to vendor-specific data without impacting standard debug functionality.

5.5 Expected Outcomes

This Specification will standardize the data structure of configuration registers and status registers, and access method. It will help datacenter customers rapidly identify system failures, precisely locate failing components, reduce root-cause analysis (RCA) time, improve hardware replacement and repair workflows, and support pre-emptive maintenance based on predicted component failure risk. The results are expected to reduce system downtime and improve the overall efficiency of the datacenter. This Specification will simplify datacenter debugging.

6 Terminology

6.1 Syntax and Conventions

6.1.1 Special Terms

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this Specification are to be interpreted as described in [BCP 14] [RFC2119] [RFC8174] when they appear.

6.1.2 Number Radix

This Specification represents some numbers in non-decimal radix. In these cases, the following radix notations are used:

4’b0001	Binary representation of the decimal value 1 explicitly expressed in 4 bits.
0x5A	Hexadecimal representation of the decimal value 90.
7’h5A	Hexadecimal representation of the decimal value 90 explicitly expressed in 7 bits.

Note:

An underscore (i.e., ’_’) is sometimes used in the representation to improve readability. For example, an underscore can be used in a Hexadecimal number to separate each four digits in the example: 0x0001_2345.

6.1.3 JSON Keys

The JSON keys defined in this Specification will use Lower Camel Case when the key name is a concatenation of multiple words. Lower Camel Case is where the first letter of the first word is in lowercase and the first letters of subsequent words are in uppercase. For example, if the key name’s description is “number of nodes”, then the key’s name will be `numberOfNodes`.

In addition to this naming convention, key names written in this Specification will use the `Courier` font.

6.2 Definitions

- **Server Fleet Operator** - A broad term for any organization that runs a large fleet of machines and provides services using that fleet. This includes Cloud Service Providers (CSP) and Hyperscalers.
- **Node** - A platform designed by a supplier integrating multiple components that run an operating system. These are sometimes also called compute nodes.
- **Life Cycle** - The operational states of a component, platform, or software product throughout its existence, such as development, validation, manufacturing, production deployment, service, and end-of-life. The life cycle state determines the availability of debug features and security restrictions.
- **Code Currency** - Describes how up-to-date, healthy, and maintainable a codebase is relative to current standards, tools, dependencies, and practices.
- **Non-Sensitive Material** - Anything that would not have any damaging effect to the owner of the material or any other entity if the material is exposed. In addition, the expectation is that exposing this material would not require any legal or any other type of consent by the owner to access the material since exposure would not harm nor be a concern of the owner.

- **Sensitive Material** - Anything that would be damaging (e.g., cause monetary loss) to the owner of the material if exposed, e.g., cryptographic keys, device identifiers, personally identifiable information, and vendor- or tenant-specific IP.
- **Local** - Data access methods designed to be collected while the collector has physical access, for example logs collected to a flash drive via the system's USB subsystem or from the PCI differential pair.
- **In-Band** - Data access methods designed to be collected over an interface between server operator's infrastructure and the OS/software running on the host. The Server Fleet Operator's infrastructure may interact with the host using the in-band management agent (i.e., software daemon/driver running on the host) by either using RAS API commands or using proprietary commands. The in-band management agent acts as a liaison and interacts with the rest of the platform in performing the requested actions. The in-band management agent may perform in-band error log collection by reading errors out of architecturally defined registers, by using firmware interfaces such as ACPI, or by using device specific drivers.
- **Out-of-Band** - Data access methods designed to be collected over interfaces that are independent of software running on the host processor. With OOB management, the BMC provides interfaces that allow external software to manage the system. For example, OOB error logging means that the BMC can access error logs, with no race conditions, and provide them to external consumers.
- **Debug Unlock** - A mechanism to send the cryptographic payload or challenge response to the device requesting that the device be put into a debug (i.e., non production) mode where debug capabilities can be enabled.

6.3 Abbreviations

DBG	Debug
e.g.	(Latin) exempli gratia - meaning: "for example"
etc.	(Latin) et cetera - meaning: "and other similar things" or "and so forth"
i.e.	(Latin) id est - meaning: "that is"

6.4 Acronyms

AMC	Accelerator Management Controller
API	Application Programming Interface
BIOS	Basic Input/Output System
BMC	Baseboard Management Controller
CI/CD	Continuous Integration and Continuous Delivery
CLI	Command-Line Interface
CPER	Common Platform Error Record
CPU	Central Processing Unit
CSP	Cloud Service Provider
DVT	Design Validation Test
EVT	Engineering Validation Test

(continued on next page)

(continued from previous page)

FW	Firmware
GPU	Graphics Processing Unit
IMA	In-band Management Agent
IP	intellectual Property
JSON	JavaScript Object Notation
JTAG	Joint Test Action Group - Bare metal interface to access on-chip debug and test resources.
MCTP	Management Component Transport Protocol
MTTD	Mean Time to Debug
NIC	Network Interconnect
NVMe	Non-Volatile Memory Express
OAM	OCP Accelerator Module
OCP	Open Compute Project
OEM	Original Equipment Manufacturer
OMA	Out-of-Band Management Agent
OOB	Out-of-Band
OS	Operating System
PCIe	Peripheral Component Interconnect Express
PLDM	Platform Level Data Model
PVT	Production Validation Test
RAS	Reliability, Availability, and Serviceability
RCA	Root Cause Analysis
SDK	Software Development Kit
SFO	Server Fleet Operator
SoC	System on Chip
SPDM	Security Protocol and Data Model
SW	Software
UBB	Universal Baseboard
USB	Universal Serial Bus
WDT	Watchdog Timer

7 Technical Overview (Informative)

7.1 Product Development Life Cycle

To better understand the requirements, usages, and restrictions on the various debug capabilities, it is important to understand the different environments and stages of progression through a product's development lifecycle. Each stage of the Product Development Life Cycle represents a specific environment that has a certain set of expectations, behaviors, and limitations on what debug can be performed. These stages form a continuous lifecycle progression. As systems move through this progression, ownership transitions from silicon providers to OEMs and ultimately to datacenter operators and tenants, while debug access progressively transitions from unrestricted hardware-level visibility toward highly controlled management-based diagnostics and telemetry-driven operations. The Product Development Life Cycle is illustrated in the diagram below.

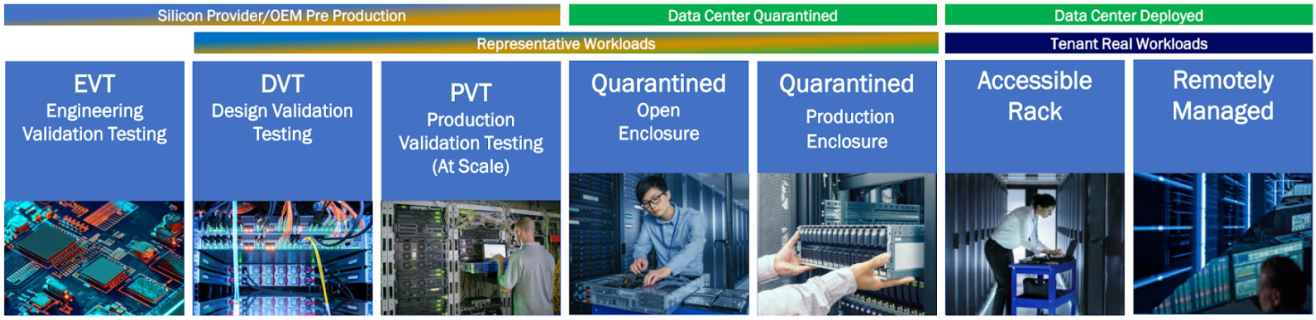


Figure 1: Product Development Life Cycle

This specification defines seven distinct debug environments that span the product lifecycle from silicon development through deployed datacenter operation. The environments differ in physical accessibility, ownership, security posture, and available debug capabilities. These stages are described in the following subsections.

It is important to note that the usual progression of the stages is from left to right in the diagram above. However, some systems could progress back from right to left due to issues that need to be resolved. For example, a system is taken out of the production environments (e.g., Remotely Managed) to move left into the Quarantined or even the Production Validation Testing (PVT) environments to debug a complicated or large issue. Once an issue is solved, the system will move again toward the right, back to the production environments again.

7.1.1 Engineering Validation Test (EVT)

The Engineering Validation Test (EVT) stage represents the earliest platform validation phase within the silicon provider or OEM development lifecycle. During EVT, engineering teams validate initial hardware, firmware, and software functionality while developing platform bring-up procedures and identifying fundamental design issues. EVT is part of the Silicon Provider/OEM pre-production phase.

The platform is typically located in a development laboratory where physical access to the system is unrestricted. Engineers can directly access the platform, open the chassis, connect physical debug tools, modify firmware, and perform invasive debug operations. Physical debug interfaces such as JTAG are available and expected to be heavily utilized during this phase.

Primary objectives during this stage include:

- Silicon bring-up
- Hardware and firmware validation
- Feature verification
- Failure discovery and characterization
- Architecture validation
- Root-cause analysis of new issues

This environment supports deep platform visibility including hardware trace, error injection, memory access, system tracing, and privileged debug capabilities. Closed-loop debug and rapid design iteration are expected.

7.1.2 Design Validation Test (DVT)

The Design Validation Test (DVT) stage follows EVT and is used to validate that the platform design satisfies functional, reliability, performance, and interoperability requirements prior to production readiness. DVT remains within the Silicon Provider/OEM pre-production phase.

Systems continue to operate in laboratory settings with extensive physical access and broad debug capabilities available. While the platform is more mature than EVT, engineering teams still need access to low-level debug mechanisms to investigate defects discovered during validation testing.

Primary objectives during this stage include:

- Design verification
- Stress and reliability testing
- Performance characterization
- Failure reproduction
- Validation of corrective actions identified during EVT

Debug workflows focus on reproducing failures, collecting detailed traces, validating fixes, and ensuring production readiness. Physical debug interfaces continue to play a significant role in root-cause analysis activities.

7.1.3 Production Validation Testing (PVT)

Production Validation Testing (PVT) stage represents the final validation stage before large-scale deployment. It is used to validate manufacturing processes, production hardware, firmware releases, and operational readiness at scale.

Systems closely resemble production configurations and are representative of customer deployments. Platform behavior is evaluated using production software, production firmware, and realistic workloads while maintaining controlled validation conditions.

Primary objectives during this stage include:

- Production readiness validation
- Manufacturing verification
- Scale testing
- Final qualification
- Discovery of issues that only appear under large-scale deployment conditions

Debug capabilities become more restricted than EVT and DVT environments, with emphasis shifting toward production-compatible mechanisms, telemetry collection, management interfaces, and automated diagnostics.

7.1.4 Quarantined: Open Enclosure

The Quarantined: Open Enclosure stage is used when a deployed platform is removed from normal service and placed into an isolated environment for detailed diagnosis and failure analysis. The chassis is physically accessible and can be opened to allow direct access to components.

The system is quarantined from tenant workloads and production operations. Engineers have physical access to the hardware and can attach debug equipment while maintaining a configuration that is representative of the deployed platform.

Primary objectives during this stage include:

- Failure replication
- Signal tracing
- Live data collection
- In-depth system analysis
- Hardware failure characterization

This environment is specifically associated with failure replication, live trace capture, and in-depth silicon and system failure analysis. This environment acts as the bridge between production systems and laboratory-level investigative debugging.

7.1.5 Quarantined: Production Enclosure

The Quarantined: Production Enclosure stage is used to investigate production failures while preserving the deployed system configuration and physical enclosure. The system is removed from normal service but remains representative of its production state.

Although isolated from tenant workloads, the platform continues to operate in a production-like configuration. Physical intrusion is minimized to avoid altering failure conditions and to maintain environmental fidelity.

Primary objectives during this stage include:

- Failure discovery
- Failure characterization
- Failure triage
- Log collection
- Root-cause investigation

This environment is associated with collection and analysis of application, operating-system, BIOS, firmware, and hardware logs to discover, characterize, and triage new failures.

7.1.6 Production: Accessible Rack

The Production: Accessible Rack stage represents a deployed datacenter platform where physical access to the rack remains possible while the platform continues to operate in a datacenter setting. This environment is part of the deployed lifecycle and typically hosts representative or real workloads.

The rack remains physically accessible to datacenter personnel while maintaining production-like behavior. Debug operations are primarily performed through management interfaces though invasive physical interfaces can be used in this environment.

Primary objectives during this stage include:

- Diagnostics of deployed systems
- Maintenance operations
- Recovery activities
- Failure prevention
- Operational optimization

This environment supports collection of telemetry, diagnostics, and serviceability data while enabling maintenance workflows that demand limited physical interaction with deployed infrastructure. It serves as an intermediate step between quarantined analysis and fully remote operation.

7.1.7 Production: - Remotely Managed

The Production: Remotely Managed stage represents the steady-state operational model for deployed datacenter systems. The platform executes tenant workloads while debug and management operations occur remotely only through standardized management interfaces.

Physical access is not assumed. Systems are managed through management networks, telemetry infrastructure, and out-of-band management mechanisms. The platform remains under strict production security controls while supporting diagnostics and recovery operations.

Primary objectives during this stage include:

- Platform monitoring
- Predictive failure analysis
- Automated diagnostics
- Maintenance
- Software and firmware updates
- Failure recovery
- Workload optimization

This environment is associated with remote maintenance, automated signature analysis, telemetry collection, trigger-based log capture, system updates, patch deployment, and recovery of known application, operating-system, and component failures as key activities in this environment. The environment also supports resident applications that optimize application, operating-system, and platform performance.

7.2 Execution Flow with Debug

It is critical that during the execution lifetime of a system, that the ability to perform progressively deeper data collection and root-cause analysis throughout the entire platform lifecycle, from boot through crash recovery, is available. A platform begins booting where firmware, hardware, and software initialization occur. Upon successful completion of boot activities, the platform transitions to the main execution state and enters normal operation. When executing, the platform is expected to deliver normal service and system functionality. If no issues are encountered, execution continues uninterrupted. However, as operational anomalies begin to emerge, the platform could enter different degraded or even halt/crash states. In degraded states, the platform remains functional but exhibits symptoms such as reduced throughput, increased latency, excessive resource consumption, or intermittent errors. If degradation worsens or corrective actions are not successful, the platform can progress to a halted/crashed (i.e., failure) state. In this condition, one or more platform functions are no longer operating correctly, and the escalation of failures could ultimately result in a platform becoming non-functional and normal execution

stops. At each stage, debug capabilities can be invoked to collect information, determine root cause, and attempt corrective action before the issue progresses to a more severe state.

The diagram below provides a generic execution flow that shows the different use cases where operational issues could occur. Note that boxes colored in green represent normal operating states, boxes in yellow represent degraded states, boxes in orange represent halted/crashed states, and boxes in blue represent debug states.

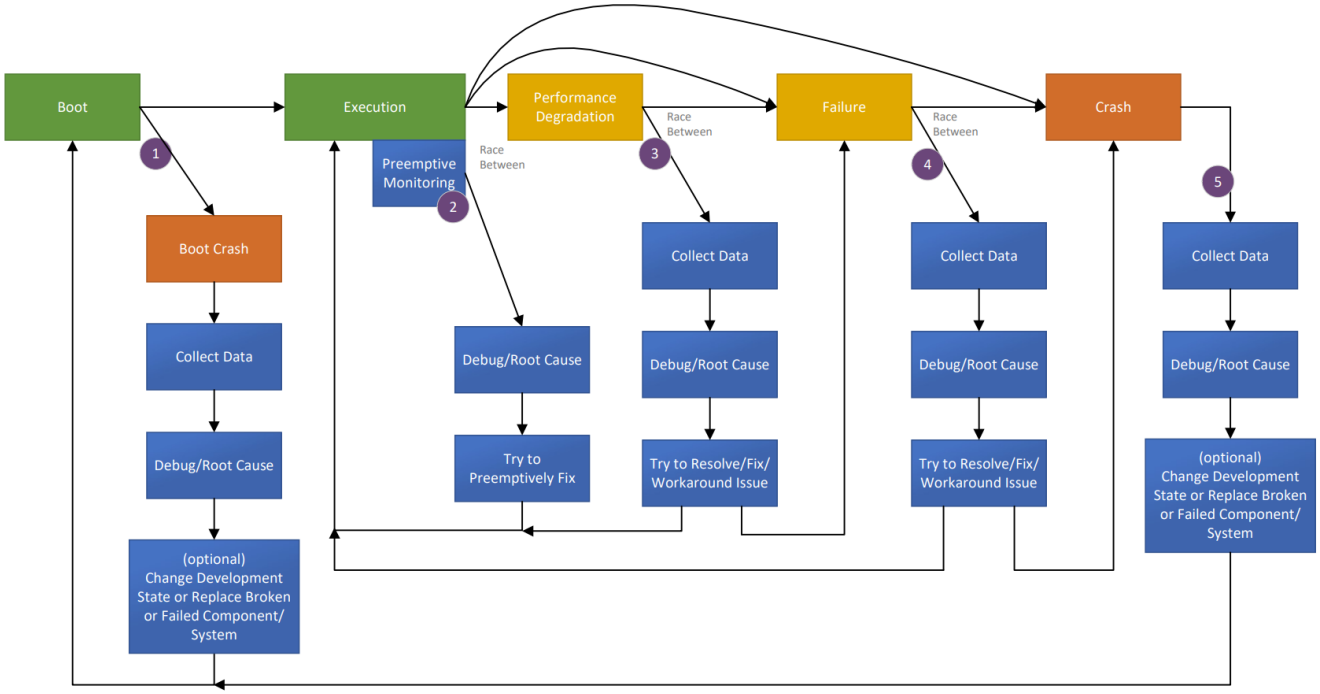


Figure 2: Execution Flow

To support this flow, the debug architecture needs to support the following:

- Pre-failure observability during normal execution.
- Data capture during degradation and failure conditions.
- Post-mortem crash analysis.
- Root-cause determination.
- Corrective action and validation.
- Return to normal operation while minimizing service disruption.

This lifecycle-oriented approach enables debug capabilities to improve platform availability, accelerate root-cause analysis, and reduce mean time to debug (MTTD) across all operational states. The main use cases, indicated by the numbered purple circles in the above diagram, are:

1. Boot Crash Handling
2. Preemptive Issues Handling
3. Performance Issues Handling
4. Failure Handling
5. Crash Handling

The next set of subsections describe these different use cases within this flow.

7.2.1 Boot Crash Handling

A boot crash occurs when the platform fails to successfully transition from the Boot state to the Execution state. The failure could occur during hardware initialization, firmware execution, security validation, configuration loading, training sequences, or other boot-time activities. This use case begins when boot execution starts and ends when the platform either enters the Execution state successfully or enters a terminal boot failure condition requiring remediation.

When a boot crash is detected:

1. The platform enters the Boot Crash state.
2. Debug capabilities are used to collect diagnostic data.
3. Root-cause analysis is performed. Verify whether the boot failure occurs consistently and determine dependency on platform configuration, software version, environmental conditions, or hardware population.
4. Corrective actions can be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.
5. The platform is rebooted and re-enters the Boot state.

It is important to note that boot-time visibility can be limited due to things such as the logging infrastructure not being available yet. In addition, security policies could restrict debug access during early boot. The recovery mechanisms will need to preserve relevant failure context.

During this use case, the following are typical debug actions taken:

- Collect boot logs.
- Capture firmware trace information.
- Retrieve crash dumps or fault records.
- Analyze initialization sequence failures.
- Identify defective hardware or firmware components.

7.2.2 Preemptive Issues Handling

This use case occurs while the platform is operating normally in the Execution state. Monitoring and telemetry mechanisms detect indicators suggesting that a future failure could occur if corrective action is not taken. Rather than waiting for visible service degradation, debug operations are initiated proactively. This use case is triggered while the platform remains functional and ends when the condition is corrected or progresses into a degradation condition.

Examples of data collected to proactively identify issues include:

- Correctable error accumulation
- Thermal margin reduction
- Resource exhaustion trends
- Reliability threshold violations
- Predictive hardware health indicators

It is important to note that detection accuracy is critical. Workload execution cannot be significantly impacted by excessive data collection. In addition, actions need to minimize disruption to production operations.

Reproducibility can be difficult because failures have not yet fully manifested. Correlation across multiple telemetry samples could be necessary.

During this use case, the following are typical debug actions taken:

- Capture health telemetry.
- Collect performance counters.
- Analyze trends and predictive indicators.
- Apply preventative mitigations or configuration changes.

7.2.3 Performance Issues Handling

Performance degradation occurs when the platform continues to function but exhibits reduced operational effectiveness. This use case begins when measurable performance reduction occurs and ends when normal operation is restored or the issue progresses to failure. For this use case, there is a race between the issue progressing toward platform failure, and the debug infrastructure collecting sufficient information to diagnose the problem.

Possible causes of performance issues include:

- Resource contention
- Hardware degradation
- Firmware inefficiencies
- Excessive error recovery activity
- Thermal or power constraints

When performance is degraded, the following steps can be taken:

1. Debug capabilities are used to collect diagnostic data.
2. Root-cause analysis is performed. Determine whether the degradation is continuous, intermittent, workload dependent, and/or environment dependent.
3. Corrective actions can be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.

It is important to note that diagnostic activities need to avoid further degrading performance. In this case, prioritization of the data collection could be needed due to resource constraints. Timely capture is important before symptoms disappear or worsen.

During this use case, the following are typical debug actions taken:

- Capture performance metrics.
- Collect traces and telemetry snapshots.
- Analyze resource utilization.
- Apply mitigations or corrective software and hardware changes.

7.2.4 Failure Handling

A failure state indicates that one or more platform functions are no longer operating correctly, although the system can still be partially operational. This use case begins when platform functionality is lost or significantly impaired and ends when either recovery occurs or the platform crashes. At this stage, there exists a race between the failure escalating leading to a crash and the debug capabilities capturing sufficient information for diagnosis and recovery.

Examples of different failures include:

- Device failures
- Link failures
- Memory subsystem failures
- Firmware service failures
- Platform service interruptions

When a failure occurs, the following steps can be taken:

1. Debug capabilities are used to collect diagnostic data.
2. Root-cause analysis is performed. Determine whether the failure is permanent, intermittent, or environment dependent.
3. Corrective actions can be taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.

It is important to note that the time available for diagnostics can be limited. In some cases, some platform services could already be unavailable. Therefore, data preservation becomes increasingly important.

During this use case, the following are typical debug actions taken:

- Capture failure records.
- Collect dump data.
- Retrieve hardware status information.
- Analyze fault isolation information.
- Apply workaround, reset, repair, or replacement actions.

7.2.5 Crash Handling

A crash represents a terminal platform condition where normal execution has stopped. The platform is unable to continue operation and recovery typically requires reset, restart, repair, or component replacement. This use case begins when normal execution terminates and ends when corrective actions have been implemented and the platform can successfully reboot.

Following a crash:

1. Crash information is collected.
2. Root-cause analysis is performed. Determine whether the crash is deterministic or sporadic.
3. Corrective actions are taken, including firmware modification, configuration updates, hardware repair or replacement, and/or component replacement.
4. The platform is rebooted for validation and recovery. Validate that the corrective action eliminates the crash condition.

It is important to note that the diagnostic information could be lost if not captured immediately. Specialized crash dump or post-mortem mechanisms could be needed for debug access. Also, security requirements will still be enforced during crash analysis.

During this use case, the following are typical debug actions taken:

- Capture crash dumps.
- Retrieve processor state.
- Collect memory contents.
- Capture hardware fault records.
- Perform post-mortem analysis.
- Identify required software, firmware, or hardware fixes.

7.3 High-Level Debug Architecture

The system debug architecture consists of in-band and out-of-band data collection paths, centralized aggregation, and analysis components.

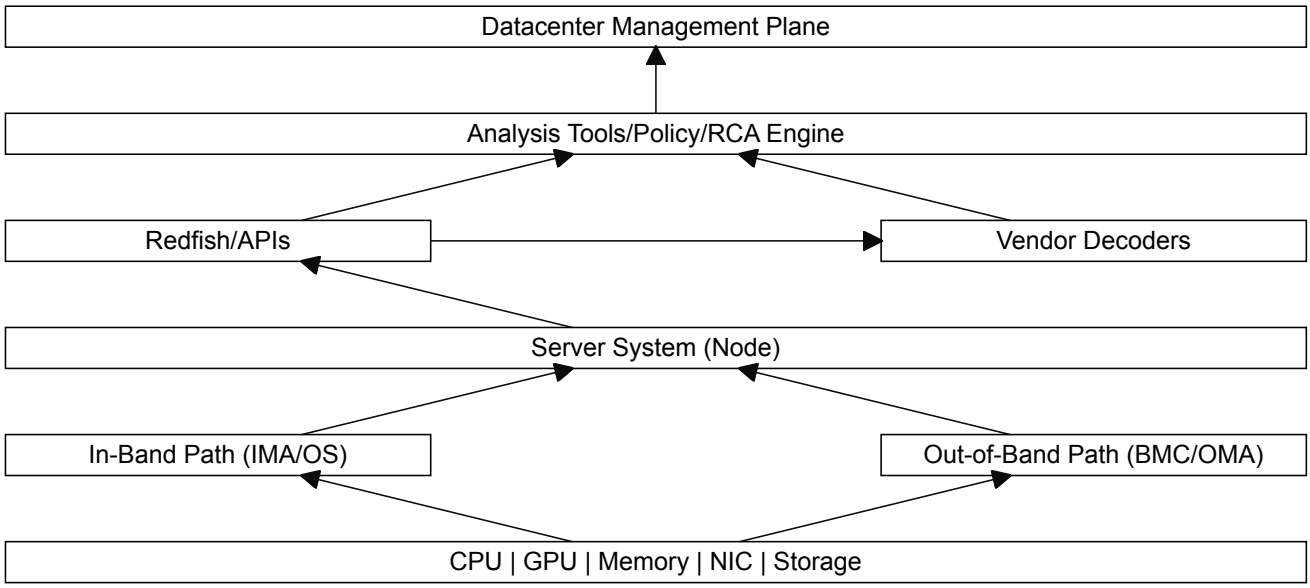


Figure 3: System Debug Architecture

Where:

- In-band path: Collects error data via OS, drivers, and kernel interfaces
- IMA: In-band Management Agent
- Out-of-band path: Collects error data independently of host software
- OMA: Out-of-band Management Agent
- Management plane: Performs analysis, correlation, and decision-making
- Vendor decoders: Translate raw hardware data into actionable insights

7.4 Configuration vs Physical Error Determination Flow

A generic end-to-end debug data flow for a given system is shown in the figure below. The remedies given at the bottom of the flow are examples and other remedies can be performed as well.

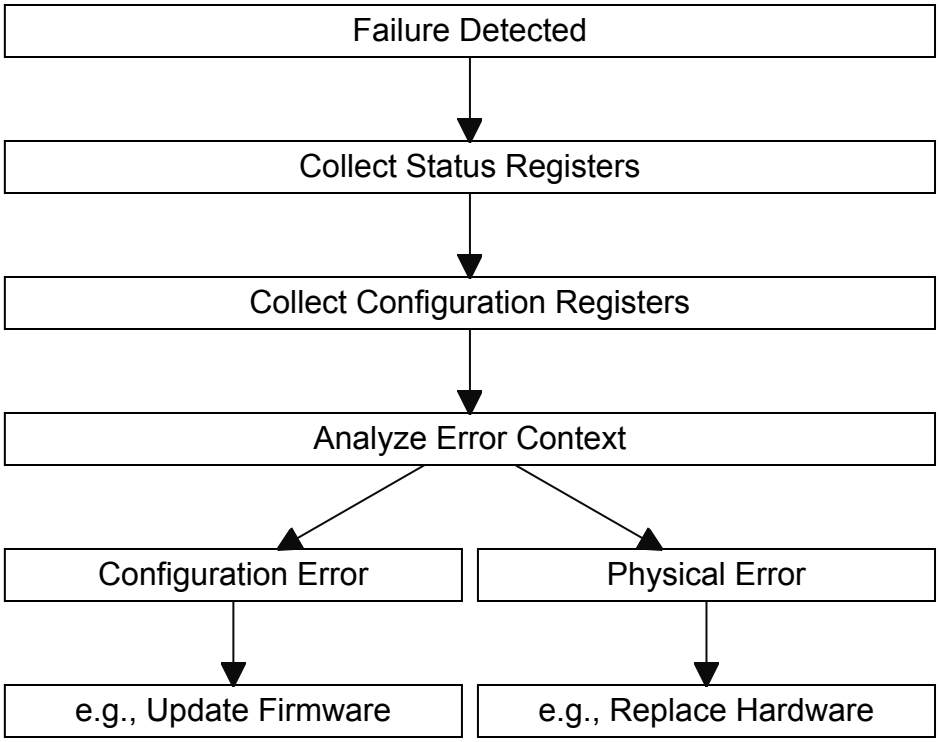


Figure 4: Configuration vs Physical Error Determination Flow

7.5 Content Arrangement

Configuration registers and status registers are primary mechanisms for device control and fault diagnosis. During system debugging, status registers are used to detect and localize failures. Configuration registers provide the settings for working operation. Failures are classified into the following categories: configuration errors, which are caused by incorrect configuration, typically resolved through firmware or driver updates, and physical errors, which are caused by hardware faults, requiring repair or replacement of components.

The system debug process includes collecting status register data to identify failing components, collecting configuration register data to identify firmware issues if there is no hardware failure, determining failure classification (configuration vs. physical), and recommending remediation actions.

The Baseboard Management Controller (BMC) acts as the central entity for system-level debugging. The BMC collects and aggregates debug data across system components, identify failing devices based on status information, determine whether a failure is hardware-related or configuration-related, and provide diagnostic information and guidance for remediation.

This Specification defines the data structures and the access methods for the necessary configuration registers and status registers of a given system. Standardized data structures and machine readable methods to retrieve all relevant information about the configuration of a given system will be discussed in Code Currency chapter. The data format and dump method of the status registers of a given system will be described in Status Dump chapter. The combination of configuration registers and status registers enable accurate fault identification and root cause analysis.

8 Security and Privacy

Debug capabilities are foundational to the development, deployment, and operation of modern datacenter platforms. Within the OCP ecosystem, debug is a critical enabler for system validation, fleet bring-up, fault isolation, reliability improvement, and root-cause analysis across heterogeneous, large-scale deployments. At the same time, debug interfaces represent one of the most powerful and sensitive control paths into a system. If not properly secured, they can undermine isolation boundaries, expose confidential and/or private tenant data, and compromise the integrity and trustworthiness of the platform. As such, security and privacy controls for debug are a core requirement for open, scalable, and trustworthy datacenter infrastructure.

Security and privacy are broad topics that have many different aspects that can be specified. Security and privacy and the necessary protections on assets, such as keys, intellectual property, and user data are important since security/privacy and debug have competing objectives. However, for this version of the Specification, the areas of security and privacy will only cover the classification of information or data. The classification of data is important when it comes to what is collected and extracted from the SoC and the system. Other aspects of security and privacy could be discussed and handled in later versions of this Specification including the necessary protections of certain classifications of data.

For the purposes of this Specification, information and data (collectively called material) that is collected and extracted from the SoC and/or system are classified into two categories: sensitive and non-sensitive.

Sensitive material, simply put, is anything that would be damaging to the owner of the material if exposed. Sensitive material sometimes is referred to as an asset and includes material such as cryptographic keys, device identifiers, and vendor- or tenant-specific IP. Due to the fact that this material could cause damage, such as monetary loss, to the owner, sensitive material needs to be handled properly in order to protect the material from being exposed to others besides the owner and authorized agents given by the owner of the material. Note, damage could be caused directly or indirectly (e.g., side channel) by the exposure of the material.

Non-sensitive material is anything that would not have any damaging effect to the owner or any other entity if the material is exposed. In other words, non-sensitive material is simply material that is not sensitive and can be exposed publicly (though the owner could choose not to expose the material for other reasons outside of security and privacy). Non-sensitive material still has value, and for purposes of this Specification, value to help debug and triage the system. However, the exposure of non-sensitive material will not harm the owner of the material and not violate any expectations or legal agreements related to the protection of sensitive material. In addition, the expectation is that exposing non-sensitive material would not require any legal or any other type of consent by the owner to access the material since exposure would not harm nor be a concern of the owner.

Future versions of this Specification will break down the sensitive material category even further into sub-categories. However, for this version of the Specification, there is currently no need. The capabilities defined in this Specification focus on the handling of non-sensitive material. In future versions where sensitive material is being handled, then this breaking down of sensitive material will be performed and detailed.

It is important to note that what is sensitive material can be dependant on who is collecting the material and/or has access to the material. For example, if a particular dump log is restricted from collection (by physical enforcement or through the system or silicon) to only the Sever Fleet Operator (SFO), then it could be allowable that sensitive data to the SFO is actually considered non-sensitive. An example of such material could be the `physicalLocation` key (and its sub-keys) in the Code Currency log. To

the SFO, this material is only accessible to members belonging to the SFO and is non-sensitive to the members within the SFO. However, outside the SFO, e.g., the system provider or silicon provider(s), this material would be sensitive. In this case, there would be mechanisms in place to ensure that only the SFO is permitted to access this dump else the material would be at risk and then no longer be considered non-sensitive. It is left to the implementer to ensure these protection mechanisms and in place. These mechanisms are not explicitly defined by this Specification. Also note that if the SFO wanted to later share this dump log (the log itself and not access to the system to collect the dump) to a system provider or silicon provider, they could strip the log of the sensitive material (e.g., remove the `physicalLocation` key and its sub-keys) from the log before transferring the log to the system provider/silicon provider. Again, a mechanism to handle this removing of the sensitive material would be necessary and is not part of this Specification.

9 Data Stacks

9.1 Code Currency

Code Currency is a term used to describe how up-to-date, healthy, and maintainable a codebase is relative to current standards, tools, dependencies, and practices. This codebase includes, but is not limited to, the following:

- Programming languages and compiler versions
- Libraries, frameworks, and SDKs
- Toolchains, build systems, and CI/CD infrastructure
- Security patches and vulnerability fixes

Code Currency refers to the timeliness, consistency, and lineage awareness of code and configuration artifacts that participate in debug enablement, control, and analysis. In the context of debug architectures, maintaining accurate code currency is essential to ensure that debug capabilities operate predictably, securely, and in alignment with platform policy across the system lifecycle.

Debug mechanisms inherently bridge development, manufacturing, validation, and field operation. As a result, debug flows often depend on a diverse set of artifacts—including firmware binaries, microcode, configuration tables, entitlement credentials, policy definitions, and tooling metadata—that evolve independently over time. Without explicit consideration of code currency, mismatches between these artifacts can introduce ambiguity in debug behavior, weaken security guarantees, and complicate auditability and incident response.

More often than not, the root cause of a failure or problem is simply that the system has not been configured correctly and/or it is running either outdated firmware, an incompatible combination of software/firmware components, or components that have not been configured appropriately. The concept of Code Currency in this Specification is used to address this issue by providing a one-stop shop standardized, machine-readable method to retrieve all relevant information about the configuration of a given system. By explicitly accounting for these forms of data covered by Code Currency, the debug architecture can reason about compatibility, enforce policy consistently, and detect invalid or unsafe combinations of code and authorization. Treating Code Currency as an architectural concern—rather than an implicit assumption—helps ensure that debug capabilities remain controlled, auditable, and resilient as the platform evolves over time.

Code currency is considered non-sensitive material which is human readable and therefore can be audited by the customer before providing it to a supplier.

9.1.1 File Format and Schema

The information for Code Currency will be exposed by a system as a file in the Redfish profile of the system. Components and subsystems will expose their part of the information to the main BMC as PLDM type 7 files. The content of the file is organized using the JSON schema defined by Requirement [CC-0001]. The relationship diagram below outlines this JSON schema.

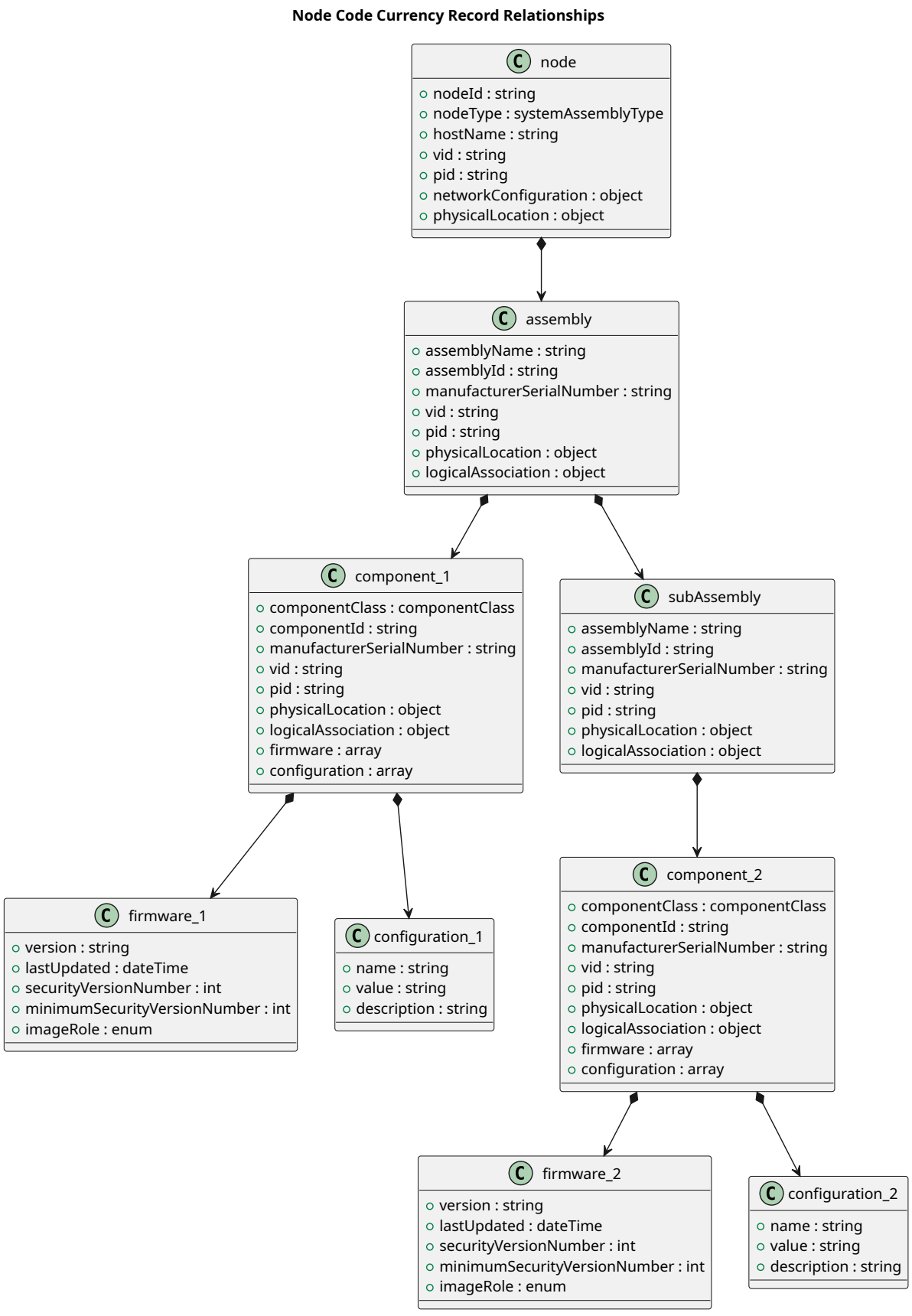


Figure 5: Code Currency Relationships

The tables below define the elements for the Code Currency JSON schema.

Table 1: Code Currency Node-Level Elements Definition

Element Name	Description
`hostName`	Hostname for network identification.
`networkConfiguration.ipAddresses`	List of IPv4 addresses. (required)
`networkConfiguration.macAddresses`	MAC addresses for network interfaces.
`networkConfiguration.ports`	Open or configured ports.
`nodeId`	Unique identifier for the node. (required)
`nodeType`	Type of node. Permitted values: `server`, `blade`, `computeNode`, `ethernetSwitch`, `networkAppliance`, `storageSystem`, `jbod`, `nvmeShelf`, `acceleratorSystem`, `gpuServer`, `managementController`, `rackManager`, `powerDistributionUnit`, `powerShelf`, `batteryBackupUnit`, `coolingDistributionUnit`, `coolingManifold`, `liquidCoolingSystem`, `chassis`, `enclosure`. (required)
`vid`	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
`pid`	PCIe Product ID represented as a four-digit hexadecimal value. (required)
`physicalLocation.datacenter`	Datacenter identifier.
`physicalLocation.rack`	Rack identifier.
`physicalLocation.row`	Row identifier.
`physicalLocation.placement.bayId`	Bay identifier if `physicalLocation.placement.type = bay`.
`physicalLocation.placement.bayPosition`	Position within the bay if `physicalLocation.placement.type = bay`.
`physicalLocation.placement.heightInU`	Height in rack units if `physicalLocation.placement.type = rackUnit`.
`physicalLocation.placement.slot`	Rack unit position if `physicalLocation.placement.type = rackUnit`.
`physicalLocation.placement.type`	Physical installation location type. Permitted values: `rackUnit`, `bay`, `slot`, `socket`, `drawer`, `shelf`, `tray`, `cage`, `zone`. (required)
`assemblies`	Array of assemblies within the node. (required)

Table 2: Code Currency assembly Elements Definition

Element Name	Description
`assemblyName`	Name of the assembly, for example `Baseboard`. (required)
`assemblyId`	Unique identifier for the assembly. (required)
`vid`	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
`pid`	PCIe Product ID represented as a four-digit hexadecimal value. (required)
`components`	Array of components contained within this assembly.
`logicalAssociation`	Logical connectivity information for the assembly.
`logicalAssociation.links`	Array of logical links associated with the assembly.
`logicalAssociation.links.connectedTo`	Identifier of the object connected to this link. (required)
`logicalAssociation.links.laneCount`	Number of lanes associated with the link.
`logicalAssociation.links.protocol`	Protocol used on the link. (required)
`manufacturerSerialNumber`	Manufacturer serial number for the assembly.
`physicalLocation.position`	Position of the assembly within the node or parent assembly.
`physicalLocation.slot`	Physical slot identifier where the assembly is installed.
`subAssemblies`	Array of nested assemblies contained within this assembly.

Table 3: Code Currency component Elements Definition

Element Name	Description
`componentClass`	Industry-aligned component class. Permitted values: `processor`, `memory`, `networkInterface`, `storageDevice`, `accelerator`, `powerSupply`, `fan`, `thermalSensor`, `voltageRegulator`, `clockGenerator`, `retimer`, `bridge`, `switchASIC`, `controller`, `bios`, `firmware`, `bmc`, `managementController`, `securityModule`, `fpga`, `cpld`. (required)
`componentId`	Unique identifier for the component. (required)
`vid`	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
`pid`	PCIe Product ID represented as a four-digit hexadecimal value. (required)

(continued on next page)

(continued from previous page)

Element Name	Description
`configuration`	Array of custom configuration attributes for the component.
`configuration.description`	Description of the configuration attribute.
`configuration.name`	Configuration attribute name. (required)
`configuration.value`	Configuration attribute value. (required)
`firmware`	Array of firmware entries associated with the component. (required)
`firmware.imageRole`	Role of the firmware image. Permitted values: `Active`, `Recovery`, `Standby`.
`firmware.lastUpdated`	Timestamp of the last firmware update.
`firmware.minimumSecurityVersionNumber`	Minimum enforced Security Version Number (SVN).
`firmware.securityVersionNumber`	Current Security Version Number (SVN).
`firmware.version`	Firmware version identifier. (required)
`logicalAssociation`	Logical connectivity information for the component.
`logicalAssociation.links`	Array of logical links associated with the component.
`logicalAssociation.links.connectedTo`	Identifier of the object connected to this link. (required)
`logicalAssociation.links.laneCount`	Number of lanes associated with the link.
`logicalAssociation.links.protocol`	Protocol used on the link. (required)
`manufacturerSerialNumber`	Manufacturer serial number for the component.
`physicalLocation.position`	Position of the component within the node or assembly.
`physicalLocation.slot`	Physical slot identifier where the component is installed.

9.1.2 Code Currency Generation

To handle diverse systems built from components and subsystems from multiple suppliers, each component containing a management endpoint exposes its own Code Currency file. The Node Management Controller and subsystem management controllers (where applicable) collect the information from the relevant components and create consolidated code currency files. The files created by subsystem management controllers will be retrieved by the Node Management Controller to expose a single file on the Redfish interface.

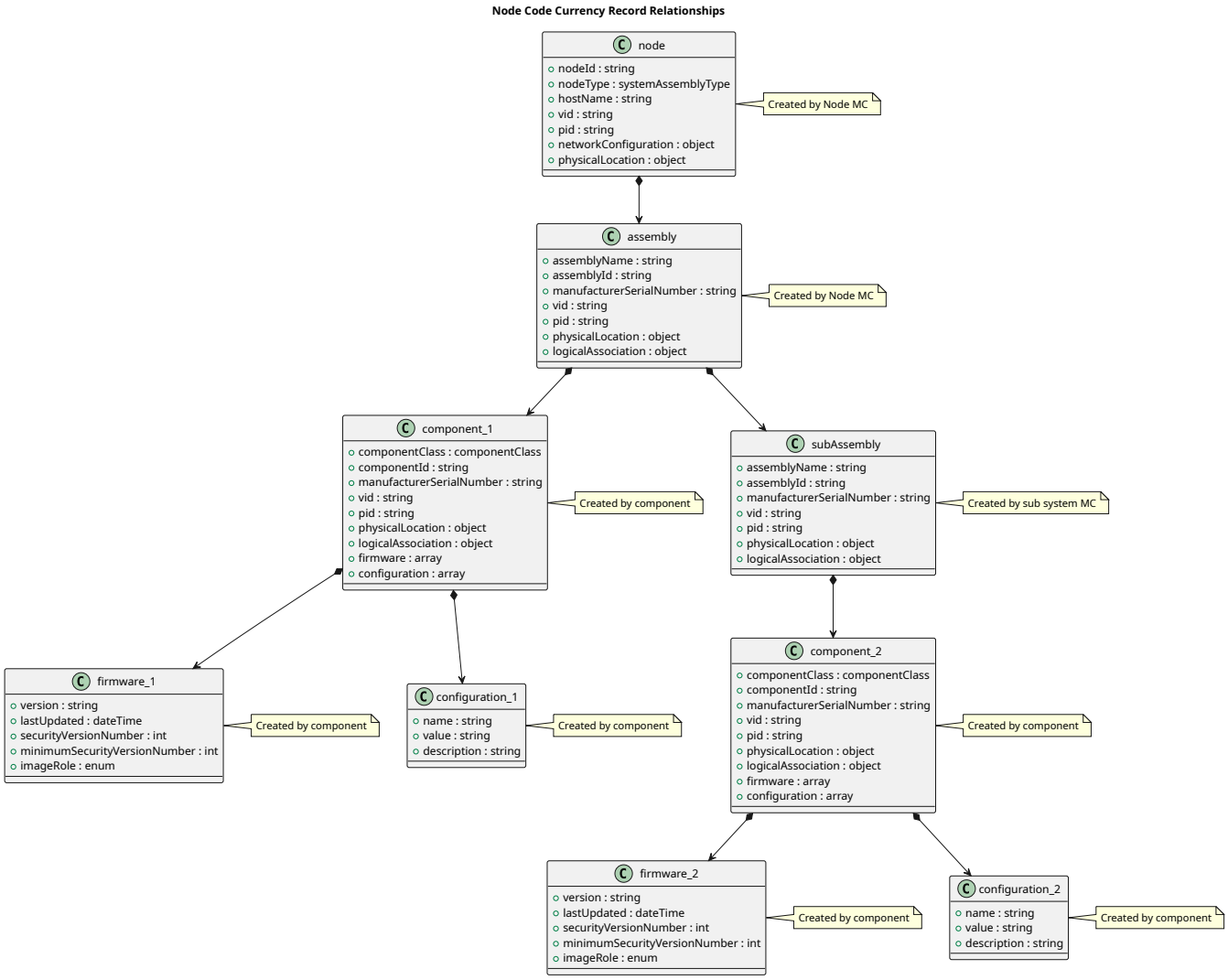


Figure 6: Code Currency Creation

9.1.3 Requirements

- **[CC-0001]** The Code Currency log SHALL be in JSON format with the following schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Node Code Currency Record",
  "type": "object",
  "properties": {
    "nodeId": {
      "type": "string",
      "description": "Unique identifier for the node"
    },
    "nodeType": {
      "$ref": "#/$defs/systemAssemblyType",
      "description": "Type of node (e.g., AI Server, Ethernet Switch, Management Tray)"
    },
```

```
    "hostName": {
      "type": "string",
      "description": "Node hostname"
    },
    "vid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Vendor ID"
    },
    "pid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Product ID"
    },
    "networkConfiguration": {
      "type": "object",
      "description": "Network configuration details for the node",
      "properties": {
        "ipAddresses": {
          "type": "array",
          "items": {
            "type": "string",
            "format": "ipv4"
          }
        },
        "ports": {
          "type": "array",
          "items": {
            "type": "integer",
            "minimum": 1,
            "maximum": 65535
          }
        },
        "macAddresses": {
          "type": "array",
          "items": {
            "type": "string",
            "pattern": "^([0-9A-Fa-f]{2}:){5}[0-9A-Fa-f]{2}$"
          }
        }
      },
      "required": [
        "ipAddresses"
      ]
    },
    "physicalLocation": {
      "type": "object",
      "description": "Physical location of the node in the datacenter",
      "properties": {
        "rack": {
          "type": "string"
        },
        "row": {
          "type": "string"
        },
        "datacenter": {
          "type": "string"
        },
        "placement": {
          "type": "object",
          "properties": {
            "type": {
              "$ref": "#/$defs/physicalLocationType",
              "description": "Placement type"
            },
            "slot": {
              "type": "integer",
              "description": "Rack unit position if RackSlot"
```

```
            },
            "heightInU": {
              "type": "integer",
              "description": "Height in U if RackSlot"
            },
            "bayId": {
              "type": "string",
              "description": "Bay identifier if Bay"
            },
            "bayPosition": {
              "type": "string",
              "description": "Position within the bay"
            }
          },
          "required": [
            "type"
          ]
        }
      },
      "required": [
        "rack",
        "row",
        "datacenter",
        "placement"
      ]
    },
    "assemblies": {
      "type": "array",
      "description": "List of assemblies within the node (supports nested assemblies)",
      "items": {
        "$ref": "#/$defs/assembly"
      }
    }
  },
  "required": [
    "nodeId",
    "nodeType",
    "vid",
    "pid",
    "assemblies"
  ],
  "$defs": {
    "assembly": {
      "type": "object",
      "properties": {
        "assemblyName": {
          "type": "string"
        },
        "assemblyId": {
          "type": "string"
        },
        "manufacturerSerialNumber": {
          "type": "string",
          "description": "Manufacturer's serial number for the assembly"
        },
        "vid": {
          "type": "string",
          "pattern": "^[0-9A-Fa-f]{4}$",
          "description": "PCIe Vendor ID"
        },
        "pid": {
          "type": "string",
          "pattern": "^[0-9A-Fa-f]{4}$",
          "description": "PCIe Product ID"
        },
        "physicalLocation": {
          "type": "object",
          "description": "Physical slot or bay where this assembly is installed",
```

```
        "properties": {
          "slot": {
            "type": "string",
            "description": "Physical slot identifier (e.g., PCIe slot, OCP slot)"
          },
          "position": {
            "type": "string",
            "description": "Position within the node or assembly"
          }
        }
      },
      "logicalAssociation": {
        "type": "object",
        "description": "Logical connectivity details for this assembly",
        "properties": {
          "links": {
            "type": "array",
            "description": "List of logical links consumed by this assembly",
            "items": {
              "type": "object",
              "properties": {
                "connectedTo": {
                  "type": "string"
                },
                "protocol": {
                  "type": "string"
                },
                "laneCount": {
                  "type": "integer"
                }
              },
              "required": [
                "connectedTo",
                "protocol"
              ]
            }
          }
        }
      },
      "components": {
        "type": "array",
        "items": {
          "$ref": "#/$defs/component"
        }
      },
      "subAssemblies": {
        "type": "array",
        "items": {
          "$ref": "#/$defs/assembly"
        }
      }
    },
    "required": [
      "assemblyName",
      "assemblyId",
      "vid",
      "pid"
    ]
  },
  "component": {
    "type": "object",
    "properties": {
      "componentClass": {
        "$ref": "#/$defs/componentClass",
        "description": "Industry-aligned component class."
      },
      "componentId": {
        "type": "string"
```

```
    },
    "manufacturerSerialNumber": {
      "type": "string",
      "description": "Manufacturer's serial number for the component"
    },
    "vid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Vendor ID"
    },
    "pid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Product ID"
    },
    "physicalLocation": {
      "type": "object",
      "description": "Physical slot or bay where this component is installed",
      "properties": {
        "slot": {
          "type": "string",
          "description": "Physical slot identifier (e.g., PCIe slot, DIMM slot)"
        },
        "position": {
          "type": "string",
          "description": "Position within the node or assembly"
        }
      }
    },
    "logicalAssociation": {
      "type": "object",
      "description": "Logical connectivity details for this component",
      "properties": {
        "links": {
          "type": "array",
          "description": "List of logical links consumed by this component",
          "items": {
            "type": "object",
            "properties": {
              "connectedTo": {
                "type": "string"
              },
              "protocol": {
                "type": "string"
              },
              "laneCount": {
                "type": "integer"
              }
            },
            "required": [
              "connectedTo",
              "protocol"
            ]
          }
        }
      }
    },
    "firmware": {
      "type": "array",
      "description": "List of firmware entries for the component",
      "items": {
        "type": "object",
        "properties": {
          "version": {
            "type": "string",
            "description": "Firmware version"
          },
          "lastUpdated": {
```

```
            "type": "string",
            "format": "date-time",
            "description": "Timestamp of last firmware update"
          },
          "securityVersionNumber": {
            "type": "integer",
            "description": "Current Security Version Number (SVN)"
          },
          "minimumSecurityVersionNumber": {
            "type": "integer",
            "description": "Minimum enforced Security Version Number (SVN)"
          },
          "imageRole": {
            "type": "string",
            "enum": [
              "Active",
              "Recovery",
              "Standby"
            ],
            "description": "Indicates if this firmware is actively running or a redundant
             ↪  recovery image"
          }
        },
        "required": [
          "version"
        ]
      }
    },
    "configuration": {
      "type": "array",
      "description": "Custom configuration attributes for the component",
      "items": {
        "type": "object",
        "properties": {
          "name": {
            "type": "string",
            "description": "Configuration attribute name"
          },
          "value": {
            "type": "string",
            "description": "Configuration attribute value"
          },
          "description": {
            "type": "string",
            "description": "Optional description of the attribute"
          }
        },
        "required": [
          "name",
          "value"
        ]
      }
    }
  },
  "required": [
    "componentClass",
    "componentId",
    "vid",
    "pid",
    "firmware"
  ]
},
"physicalLocationType": {
  "type": "string",
  "description": "Industry-aligned physical installation location type.",
  "enum": [
    "rackUnit",
    "bay",
```

```
          "slot",
          "socket",
          "drawer",
          "shelf",
          "tray",
          "cage",
          "zone"
        ]
      },
      "componentClass": {
        "type": "string",
        "description": "Industry-aligned component class (inventory category).",
        "enum": [
          "processor",
          "memory",
          "networkInterface",
          "storageDevice",
          "accelerator",
          "powerSupply",
          "fan",
          "thermalSensor",
          "voltageRegulator",
          "clockGenerator",
          "retimer",
          "bridge",
          "switchASIC",
          "controller",
          "bios",
          "firmware",
          "bmc",
          "managementController",
          "securityModule",
          "fpga",
          "cpld"
        ]
      },
      "systemAssemblyType": {
        "type": "string",
        "description": "Industry-aligned top-level assembly type (system/chassis class).",
        "enum": [
          "server",
          "blade",
          "computeNode",
          "ethernetSwitch",
          "networkAppliance",
          "storageSystem",
          "jbod",
          "nvmeShelf",
          "acceleratorSystem",
          "gpuServer",
          "managementController",
          "rackManager",
          "powerDistributionUnit",
          "powerShelf",
          "batteryBackupUnit",
          "coolingDistributionUnit",
          "coolingManifold",
          "liquidCoolingSystem",
          "chassis",
          "enclosure"
        ]
      }
    }
  }
}
```

- **[CC-0002]** The Code Currency log SHALL only contain non-sensitive material. Typically, this means that the data collected in the log does not contain any Server Fleet Operator sensitive data,

end-user sensitive data, or data collected directly from the workload that was executing. See the Security and Privacy Section for specific details in what is non-sensitive material. This allows the Code Currency log to be collected in a production, mission mode (i.e., with real workloads) including in confidential compute environments.

- **[CC-0003]** The Code Currency log SHALL be readable and comprehensible to everyone permitted access to the system based on Requirement [CC-0002]. If not in plain, human-readable text (in part or in full), then the user who obtained the Code Currency log SHALL be able to decode the log. If encrypted (in part or in full), then the user who obtained the Code Currency log SHALL be able to decrypt the log.
- **[CC-0004]** The Code Currency log SHOULD be persistent across resets and/or be stored in non-volatile storage. This is to allow collection of the log when access is blocked due to an error or crash.
- **[CC-0005]** The collection of the Code Currency log SHALL be triggered automatically, when enabled, at the node level upon detecting the crash state without dependencies on the management/control plane.
- **[CC-0006]** The collection of the Code Currency log MAY also be triggered on other errors or events. These errors/events are implementation-defined and not defined by this Specification.
- **[CC-0007]** The Code Currency feature SHALL have a survivability method to recover from errors or hangs in the data collection procedure. This can include timeouts or retries.
- **[CC-0008]** The Code Currency feature SHALL have a mechanism to enable and disable the feature. Since the Code Currency log only contains non-sensitive material, the feature MAY be enabled by default.
- **[CC-0009]** The Code Currency feature SHALL be enabled without the need to authenticate or perform a debug unlock to place the device into a debug mode.
- **[CC-0010]** Each component containing a management controller endpoint SHALL expose its own Code Currency log as a PLDM Type 7 file.
- **[CC-0011]** Each node or subsystem management controller SHALL collect the information from the relevant components and expose consolidated Code Currency log.
- **[CC-0012]** The hierarchical collection of Code Currency data and/or log SHOULD follow the physical design of the node/system.
- **[CC-0013]** The existing local management controller SHOULD be used to aggregate the hierarchical collection of the Code Currency data and/or log.
- **[CC-0014]** The maximum size of an aggregated Code Currency log for a node (visible from Redfish) SHOULD not exceed 1 MBytes.
- **[CC-0015]** The maximum time for the topmost management controller of a node to pull an aggregated Code Currency log (visible from Redfish) SHOULD not exceed 1 second.

9.2 Status Dump

A Status Dump is used to provide a way to quickly retrieve relevant debug information about the environment for a given system and its components. A Status Dump log contains state data and error codes from a system or component intended to assist in debugging and troubleshooting device fault

states. The data of the Status Dump describes the component as a part of the environment that it exists in.

9.2.1 Data Selection Methodology

The Status Dump data model is intended to provide a concise, interoperable, and actionable set of diagnostic information that enables rapid fault isolation and system troubleshooting while minimizing implementation burden on platform components. The selection of Status Dump fields/keys was guided by the objective of capturing the minimum information required to identify the source of a fault, determine the current operational state of a component, and understand the most recent error condition without requiring service interruption or specialized vendor tooling.

The mandatory set of Status Dump fields represents the minimum viable data required to provide meaningful debug value across a broad range of platform components. These required fields/keys were selected based on the following principles:

- **Component Identification** - Every dump will contain sufficient identifying information to determine the originating component and its function within the system. This allows collected data to be correlated across heterogeneous devices and enables automated processing by fleet management and service tools.
- **Operational Availability** - Required data will remain accessible during conditions such as CPU lockup events and will be retrievable without requiring platform downtime or intrusive debug operations.
- **Accessibility and Auditability** - Required fields will be represented in a format that can be interpreted using publicly accessible tools or otherwise be directly human readable. This ensures that dump data remains useful to the hardware owner regardless of the component vendor.
- **Current Fault Visibility** - The mandatory dataset includes information describing the most recent error condition, whether represented by an error code, status indicator, or corruption indication. This information provides the baseline context necessary for root-cause analysis.
- **Component-Level Diagnostics** - The mandatory dataset focuses on information directly associated with the reporting component, ensuring that each component can independently communicate its health and failure status.
- **Low-Impact Collection** - Required fields are limited to information that can be collected without measurable impact to normal system operation and can be retrieved on demand by the user.

In addition to this mandatory set, there is an additional recommended set of Status Dump fields/keys that contain data elements that provide additional diagnostic value but might not be available on all implementations due to architectural, cost, performance, security, or proprietary technology constraints. These fields/keys are therefore considered recommended when available, but each field/key is not required for compliance. These recommended fields/keys were selected according to the following principles:

- **Enhanced Diagnostic Depth** - Information such as historical error logs, extended debug data, and vendor-specific diagnostic content can significantly improve root-cause determination but often requires implementation-specific infrastructure or proprietary tooling.
- **Improved Serviceability** - Data that supports precise component localization, such as physical socket identifiers or other topology information, improves operational efficiency but might not be universally supported across all device classes.
- **Operational Automation** - Features such as alert generation, scheduled dump creation, dump lifecycle management, and summary reporting support large-scale datacenter operations but are

not fundamental to the basic Status Dump capability.
- **Advanced Collection Mechanisms** - Some diagnostic information might require collection methods that introduce performance overhead or rely on out-of-band management infrastructure. Such information provides incremental value but is intentionally separated from the mandatory dataset to preserve broad applicability and low-impact operation.

By separating the Status Dump schema into mandatory and recommended categories, this Specification establishes a consistent baseline that all compliant implementations can support while providing a path for vendors to expose richer diagnostic information when platform capabilities permit. This approach promotes interoperability, scalability, and ease of deployment while maximizing the usefulness of diagnostic data across a diverse ecosystem of system components.

9.2.2 File Format and Schema

The Status Dump will be exposed by a system as a file in the Redfish profile of the system. Components and subsystems will expose their part of the information to the main BMC as PLDM type 7 files. The content of the file is organized using the JSON schema defined by Requirement [SD-0001].

The tables below define the elements of the Status Dump JSON schema.

Table 4: Status Dump Node-Level Elements Definition

Element Name	Description
`nodeId`	Unique identifier for the node. (required)
`nodeType`	Type of node. Permitted values: `server`, `blade`, `computeNode`, `ethernetSwitch`, `networkAppliance`, `storageSystem`, `jbod`, `nvmeShelf`, `acceleratorSystem`, `gpuServer`, `managementController`, `rackManager`, `powerDistributionUnit`, `powerShelf`, `batteryBackupUnit`, `coolingDistributionUnit`, `coolingManifold`, `liquidCoolingSystem`, `chassis`, `enclosure`. (required)
`hostName`	Hostname for network identification.
`vid`	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
`pid`	PCIe Product ID represented as a four-digit hexadecimal value. (required)
`healthy`	Indicates whether the node is healthy. (required)
`mode`	Operational mode of the node. (required)
`status`	Current status of the node. (required)
`lastError`	Most recent error encountered by the node.
`errorLog`	Array of error log entries.
`lifecycle`	Lifecycle state of the node. (required)
`power`	Power telemetry object.

(continued on next page)

(continued from previous page)

Element Name	Description
power.power	Power reading. (required)
power.powerUnit	Power unit. Permitted values: Watt, kiloWatt, megaWatt.
power.voltage	Voltage reading. (required)
power.voltageUnit	Voltage unit. Permitted values: Volt, kiloVolt.
power.current	Current reading. (required)
power.currentUnit	Current unit. Permitted values: Amps, kiloAmps, megaAmps.
temperature.temperature	Temperature reading. (required)
temperature.temperatureUnit	Temperature unit. Permitted values: Degrees C, Degrees F, Kelvins.
vendor.organization	Vendor-defined organization name.
physicalLocation.datacenter	Datacenter identifier.
physicalLocation.rack	Rack identifier.
physicalLocation.row	Row identifier.
physicalLocation.placement.type	Physical installation location type. Permitted values: rackUnit, bay, slot, socket, drawer, shelf, tray, cage, zone. (required)
physicalLocation.placement.slot	Rack unit position if type is rackUnit.
physicalLocation.placement.heightInU	Height in rack units if type is rackUnit.
physicalLocation.placement.bayId	Bay identifier if type is bay.
physicalLocation.placement.bayPosition	Position within the bay if type is bay.
assemblies	Array of assemblies contained within the node. (required)

Table 5: Status Dump assembly Elements Definition

Element Name	Description
assemblyName	Name of the assembly. (required)
assemblyId	Unique identifier for the assembly. (required)
manufacturerSerialNumber	Manufacturer serial number for the assembly.
vid	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
pid	PCIe Product ID represented as a four-digit hexadecimal value. (required)
healthy	Indicates whether the assembly is healthy. (required)
mode	Operational mode of the assembly.

(continued on next page)

(continued from previous page)

Element Name	Description
`status`	Current status of the assembly.
`lastError`	Most recent error for the assembly.
`errorLog`	Array of error log entries.
`lifecycle`	Lifecycle state of the assembly.
`power`	Power telemetry object.
`temperature`	Temperature telemetry object.
`vendor`	Vendor-defined data.
`physicalLocation.slot`	Physical slot identifier where the assembly is installed.
`physicalLocation.position`	Position of the assembly within the node or parent assembly.
`components`	Array of components contained within the assembly.
`subAssemblies`	Array of nested assemblies.

Table 6: Status Dump component Elements Definition

Element Name	Description
`componentClass`	Industry-aligned component class. Permitted values: `processor`, `memory`, `networkInterface`, `storageDevice`, `accelerator`, `powerSupply`, `fan`, `thermalSensor`, `voltageRegulator`, `clockGenerator`, `retimer`, `bridge`, `switchASIC`, `controller`, `bios`, `firmware`, `bmc`, `managementController`, `securityModule`, `fpga`, `cpld`.
`componentId`	Unique identifier for the component. (required)
`manufacturerSerialNumber`	Manufacturer serial number for the component.
`vid`	PCIe Vendor ID represented as a four-digit hexadecimal value. (required)
`pid`	PCIe Product ID represented as a four-digit hexadecimal value. (required)
`physicalLocation.slot`	Physical slot identifier where the component is installed.
`physicalLocation.position`	Position of the component within the node or assembly.
`healthy`	Indicates whether the component is healthy. (required)
`mode`	Operational mode of the component.
`status`	Current status of the component. (required)

(continued on next page)

(continued from previous page)

Element Name	Description
`lastError`	Most recent error encountered by the component. (required)
`errorLog`	Array of error log entries.
`lifecycle`	Lifecycle state of the component.
`power`	Power telemetry object.
`temperature`	Temperature telemetry object.
`vendor.organization`	Vendor-defined organization name.
`utilization`	Utilization value or utilization object.
`utilization.accelerator`	Accelerator utilization.
`utilization.acceleration`	Acceleration utilization.
`utilization.ram`	RAM utilization.
`clockSpeed`	Clock speed reading.
`clockSpeedUnit`	Clock speed unit (for example GHz or MHz).
`interface`	Physical or logical interface type.
`communication`	Communication protocol used by the component.
`dataRate`	Data transfer rate.
`dataRateValue`	Data rate unit descriptor.
`performance.Packet Loss/Resend`	Packet loss or resend count/rate.
`performance.Thermal Throttling`	Thermal throttling metric.
`performance.ECC`	ECC-related metric.
`latency`	Latency measurement.
`latencyUnit`	Latency measurement unit.
`errorRate`	Error rate measurement.
`errorUnit`	Error rate unit.
`subComponents`	Array of nested sub-components.

9.2.3 Status Dump Generation

To handle diverse systems built from components and subsystems from multiple suppliers, each component containing a management endpoint exposes its own Status Dump file. The Node Management Controller and subsystem management controllers (where applicable) collect the information from the relevant components and create consolidated Status Dump files. The files created by subsystem management controllers will be retrieved by the Node Management Controller to expose a single file on the Redfish interface.

The relationship matches what is depicted in the Code Currency Section.

9.2.4 Requirements

- **[SD-0001]** The Status Dump log SHALL be in JSON format with the following schema:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "OCP System Status Dump Schema",
  "type": "object",
  "properties": {
    "nodeId": {
      "type": "string",
      "description": "Unique identifier for the node"
    },
    "nodeType": {
      "$ref": "#/$defs/systemAssemblyType",
      "description": "Type of node (e.g., AI Server, Ethernet Switch, Management Tray)"
    },
    "hostName": {
      "type": "string",
      "description": "Node hostname"
    },
    "vid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Vendor ID"
    },
    "pid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Product ID"
    },
    "healthy": {
          "type": "boolean"
        },
    "mode": {
      "type": "string"
    },
    "status": {
      "type": "string"
    },
    "lastError": {
      "type": "string",
      "examples": ["NULL", "1D10T", "501", "T"]
    },
    "errorLog": {
      "type": "array",
      "items": { "type": "string" }
    },
    "lifecycle": {
      "type": "string"
    },
    "power": {
      "$ref": "#/$defs/power"
    },
    "temperature": {
      "$ref": "#/$defs/temperature"
    },
    "vendor": {
      "$ref": "#/$defs/vendor"
    },
    "physicalLocation": {
      "type": "object",
      "description": "Physical location of the node in the datacenter",
      "properties": {
        "rack": {
          "type": "string"
        },
        "row": {
          "type": "string"
        },
        "datacenter": {
          "type": "string"
```

```
        },
        "placement": {
          "type": "object",
          "properties": {
            "type": {
              "$ref": "#/$defs/physicalLocationType",
              "description": "Placement type"
            },
            "slot": {
              "type": "integer",
              "description": "Rack unit position if RackSlot"
            },
            "heightInU": {
              "type": "integer",
              "description": "Height in U if RackSlot"
            },
            "bayId": {
              "type": "string",
              "description": "Bay identifier if Bay"
            },
            "bayPosition": {
              "type": "string",
              "description": "Position within the bay"
            }
          },
          "required": [
            "type"
          ]
        }
      },
      "required": [
        "rack",
        "row",
        "datacenter",
        "placement"
      ]
    },
    "assemblies": {
      "type": "array",
      "description": "List of assemblies within the node (supports nested assemblies)",
      "items": {
        "$ref": "#/$defs/assembly"
      }
    }
  },
  "required": [
    "nodeId",
    "nodeType",
    "vid",
    "pid",
    "healthy",
    "mode",
    "status",
    "lifecycle",
    "assemblies"
  ],
  "$defs": {
    "power": {
      "type": "object",
      "required": ["power","voltage", "current"],
      "additionalProperties": false,
      "properties": {
        "power": {
          "type": "number"
         },
        "powerUnit": {
          "type": "string",
          "enum" : [
```

```
          "Watt",
          "kiloWatt",
          "megaWatt"
        ],
         "default": "Watt"
      },
      "voltage": {
        "type": "number"
      },
      "voltageUnit": {
        "type": "string",
        "enum" : [
          "Volt",
          "kiloVolt"
        ],
        "default": "Volt"
      },
      "current": {
        "type": "number"
      },
      "currentUnit": {
        "type": "string",
        "enum" : [
          "Amps",
          "kiloAmps",
          "megaAmps"
        ],
        "default": "Amps"
      }
    }
  },
  "temperature": {
    "type": "object",
    "required": ["temperature"],
    "additionalProperties": false,
    "properties": {
      "temperature": {
        "type": "number"
      },
      "temperatureUnit": {
        "type": "string",
        "enum" : [
          "Degrees C",
          "Degrees F",
          "Kelvins"
        ],
         "default": "Degrees C"
      }
    }
  },
  "performance": {
    "type": "object",
    "description": "Performance metrics; keys vary by component type.",
    "additionalProperties": {
      "type": "string"
    },
    "properties": {
      "Packet Loss/Resend": {
        "type": "number"
      },
      "Thermal Throttling": {
        "type": "number"
      },
      "ECC": {
        "type": "number"
      }
    }
  },
```

```
"utilization": {
  "description": "Utilization may be a single percentage string or a map of resource
  ↪  utilizations.",
  "oneOf": [
    { "type": "string" },
    {
      "type": "object",
      "additionalProperties": {
        "type": "number"
      },
      "properties": {
        "accelerator": {
          "type": "number"
          },
        "acceleration": {
           "type": "number"
         },
        "ram": {
          "type": "number"
         }
      }
    }
  ]
},
"vendor": {
  "description": "Vendor defined details when present; otherwise null.",
  "oneOf": [
    { "type": "null" },
    {
      "type": "object",
      "additionalProperties": {
        "type": "string"
      },
      "properties": {
        "organization": { "type": "string" }
      }
    }
  ]
},
"assembly": {
  "type": "object",
  "properties": {
    "assemblyName": {
      "type": "string"
    },
    "assemblyId": {
      "type": "string"
    },
    "manufacturerSerialNumber": {
      "type": "string",
      "description": "Manufacturer's serial number for the assembly"
    },
    "vid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Vendor ID"
    },
    "pid": {
      "type": "string",
      "pattern": "^[0-9A-Fa-f]{4}$",
      "description": "PCIe Product ID"
    },
    "healthy": {
      "type": "boolean"
    },
    "mode": {
      "type": "string"
    },
```

```
      "status": {
        "type": "string"
      },
      "lastError": {
        "type": "string",
        "examples": ["NULL", "1D10T", "501", "T"]
      },
      "errorLog": {
        "type": "array",
        "items": { "type": "string" }
      },
      "lifecycle": {
        "type": "string"
      },
      "power": {
        "$ref": "#/$defs/power"
      },
      "temperature": {
        "$ref": "#/$defs/temperature"
      },
      "vendor": {
        "$ref": "#/$defs/vendor"
      },
      "physicalLocation": {
        "type": "object",
        "description": "Physical slot or bay where this assembly is installed",
        "properties": {
          "slot": {
            "type": "string",
            "description": "Physical slot identifier (e.g., PCIe slot, OCP slot)"
          },
          "position": {
            "type": "string",
            "description": "Position within the node or assembly"
          }
        }
      },
      "components": {
        "type": "array",
        "items": {
          "$ref": "#/$defs/component"
        }
      },
      "subAssemblies": {
        "type": "array",
        "items": {
          "$ref": "#/$defs/assembly"
        }
      }
    },
    "required": [
      "assemblyName",
      "assemblyId",
      "vid",
      "pid",
      "healthy"
    ]
  },
  "component": {
    "type": "object",
    "properties": {
      "componentClass": {
        "$ref": "#/$defs/componentClass",
        "description": "Industry-aligned component class."
      },
      "componentId": {
        "type": "string"
      },
```

```
"manufacturerSerialNumber": {
  "type": "string",
  "description": "Manufacturer's serial number for the component"
},
"vid": {
  "type": "string",
  "pattern": "^[0-9A-Fa-f]{4}$",
  "description": "PCIe Vendor ID"
},
"pid": {
  "type": "string",
  "pattern": "^[0-9A-Fa-f]{4}$",
  "description": "PCIe Product ID"
},
"physicalLocation": {
  "type": "object",
  "description": "Physical slot or bay where this component is installed",
  "properties": {
    "slot": {
      "type": "string",
      "description": "Physical slot identifier (e.g., PCIe slot, DIMM slot)"
    },
    "position": {
      "type": "string",
      "description": "Position within the node or assembly"
    }
  }
},
"healthy": {
  "type": "boolean"
},
"mode": { "type": "string" },
"status": {
  "type": "string",
  "examples": ["Ok", "Crash", "ERD"]
},
"lastError": {
  "type": "string",
  "examples": ["NULL", "1D10T", "501", "T"]
},
"errorLog": {
  "type": "array",
  "items": {
    "type": "string"
  }
},
"lifecycle": {
  "type": "string",
  "examples": ["production", "quarantine", "development"]
},
"power": {
  "$ref": "#/$defs/power"
},
"temperature": {
  "$ref": "#/$defs/temperature"
},
"vendor": {
  "$ref": "#/$defs/vendor"
},
"utilization": {
  "$ref": "#/$defs/utilization"
},
"clockSpeed": {
  "type": "number"
},
"clockSpeedUnit": {
  "type": "string",
  "examples": ["GHz", "MHz"]
```

```
      },
      "interface": {
        "type": "string",
        "examples": ["USB", "PCI-E"]
      },
      "communication": {
        "type": "string",
        "examples": ["PLDM", "Redfish"]
      },
      "dataRate": {
        "type": "number"
      },
      "dataRateValue": {
        "type": "string",
        "examples": ["MT/s"]
      },
      "performance": {
        "$ref": "#/$defs/performance"
      },
      "latency": {
        "type": "number"
      },
      "latencyUnit": {
        "type": "string",
        "examples": ["Seconds", "Milliseconds"]
      },
      "errorRate": {
        "type": "number"
       },
      "errorUnit": {
        "type": "string",
        "examples": ["errors per second"]
      },
      "subComponents": {
        "type": "array",
        "items": {
          "$ref": "#/$defs/component"
        }
      }
    },
    "required": [
      "componentId",
      "vid",
      "pid",
      "healthy",
      "status",
      "lastError"
    ]
  },
  "physicalLocationType": {
    "type": "string",
    "description": "Industry-aligned physical installation location type.",
    "enum": [
      "rackUnit",
      "bay",
      "slot",
      "socket",
      "drawer",
      "shelf",
      "tray",
      "cage",
      "zone"
    ]
  },
  "componentClass": {
    "type": "string",
    "description": "Industry-aligned component class (inventory category).",
    "enum": [
```

```
          "processor",
          "memory",
          "networkInterface",
          "storageDevice",
          "accelerator",
          "powerSupply",
          "fan",
          "thermalSensor",
          "voltageRegulator",
          "clockGenerator",
          "retimer",
          "bridge",
          "switchASIC",
          "controller",
          "bios",
          "firmware",
          "bmc",
          "managementController",
          "securityModule",
          "fpga",
          "cpld"
        ]
      },
      "systemAssemblyType": {
        "type": "string",
        "description": "Industry-aligned top-level assembly type (system/chassis class).",
        "enum": [
          "server",
          "blade",
          "computeNode",
          "ethernetSwitch",
          "networkAppliance",
          "storageSystem",
          "jbod",
          "nvmeShelf",
          "acceleratorSystem",
          "gpuServer",
          "managementController",
          "rackManager",
          "powerDistributionUnit",
          "powerShelf",
          "batteryBackupUnit",
          "coolingDistributionUnit",
          "coolingManifold",
          "liquidCoolingSystem",
          "chassis",
          "enclosure"
        ]
      }
    }
  }
```

- **[SD-0002]** The Status Dump log SHALL only contain non-sensitive material. Typically, this means that the data collected in the log does not contain any Server Fleet Operator sensitive data, end-user sensitive data, or data collected directly from the workload that was executing. See the Security and Privacy Section for specific details in what is non-sensitive material. This allows the Status Dump log to be collected in a production, mission mode (i.e., with real workloads) including in confidential compute environments.
- **[SD-0003]** The Status Dump log SHALL be readable and comprehensible to everyone permitted access to the system based on Requirement [SD-0002]. If not in plain, human-readable text (in part or in full), then the user who obtained the Status Dump log SHALL be able to decode the log. If encrypted (in part or in full), then the user who obtained the Status Dump log SHALL be able to decrypt the log.

- **[SD-0004]** The Status Dump log SHOULD be persistent across resets and/or be stored in non-volatile storage. This is to allow collection of the log when access is blocked due to an error or crash.
- **[SD-0005]** The collection of the Status Dump log SHALL be triggered automatically, when enabled, at the node level upon detecting the crash state without dependencies on the management/control plane.
- **[SD-0006]** The collection of the Status Dump log MAY also be triggered on other errors or events. These errors/events are implementation-defined and not defined by this Specification.
- **[SD-0007]** The Status Dump feature SHALL have a survivability method to recover from errors or hangs in the data collection procedure. This can include timeouts or retries.
- **[SD-0008]** The Status Dump feature SHALL have a mechanism to enable and disable the feature. Since the Status Dump log only contains non-sensitive material, the feature MAY be enabled by default.
- **[SD-0009]** The Status Dump feature SHALL be enabled without the need to authenticate or perform a debug unlock to place the device into a debug mode.
- **[SD-0010]** Each component containing a management controller endpoint SHALL expose its own Status Dump log as a PLDM Type 7 file.
- **[SD-0011]** Each node or subsystem management controller SHALL collect the information from the relevant components and expose consolidated Status Dump log.
- **[SD-0012]** The hierarchical collection of Status Dump data and/or log SHOULD follow the physical design of the node/system.
- **[SD-0013]** The existing local management controller SHOULD be used to aggregate the hierarchical collection of the Status Dump data and/or log.
- **[SD-0014]** The maximum size of an aggregated Status Dump log for a node (visible from Redfish) SHOULD not exceed 1 MBytes.
- **[SD-0015]** The maximum time for the topmost management controller of a node to pull an aggregated Status Dump log (visible from Redfish) SHOULD not exceed 1 second.

10 Future Work

Future versions of this Specification will include the following topics, requirements, and/or capabilities.

- Describe the security and privacy protections on sensitive material allowing the proper access requirements for debug capabilities.
- Add sub-categories of sensitive material and the associated requirements and necessary protections.
- Describe how to securely communicate/transfer debug data across interfaces on the platform. (i.e., secure debug communication)
- Describe how to communicate/transfer console data messages (addition to the Data Stacks Section).
- Describe how to interactively perform debug and run control (addition to the Data Stacks Section).
- Describe the layering of the capabilities and how to implement in the “stack” of the datacenter.
- Describe how to handle multiple users debugging within the datacenter (mostly at the node level is where it is interesting and needs a common method).
- Describe how to detect and handle performance degradation.
- Describe generic versions of the debug methodologies created by the CPU and GPU closed CLA Workstreams.
- Describe methodologies to utilize AI to aid and accelerate debug.

Appendix A: Code Currency Examples

Below is a JSON example for Code Currency of a hypothetical node with:

- Dual-socket CPUs
- Memory modules
- BMC
- Two NVMe drives
- Two NICs on a hot-swap backplane
- UBB subassembly with BMC
- Two OAM subassemblies (each with GPU and AMC)


```
{
  "nodeId": "AI-Node-001",
  "nodeType": "gpuServer",
  "hostName": "ai-node-001.dc.local",
  "vid": "0CC0",
  "pid": "0001",
  "networkConfiguration": {
    "ipAddresses": [
      "192.168.10.101"
    ],
    "ports": [
      22,
      443
    ],
    "macAddresses": [
      "00:1A:2B:3C:4D:5E"
    ]
  },
  "physicalLocation": {
    "rack": "R42",
    "row": "Row-A",
    "datacenter": "DC-West",
    "placement": {
      "type": "rackUnit",
      "slot": 12,
      "heightInU": 2
    }
  },
  "assemblies": [
    {
      "assemblyName": "Baseboard",
      "assemblyId": "BB-001",
      "manufacturerSerialNumber": "BB-SN-12345",
      "vid": "0CC0",
      "pid": "0002",
      "components": [
        {
          "componentClass": "processor",
          "componentId": "CPU-001",
          "manufacturerSerialNumber": "CPU-SN-11111",
          "vid": "8086",
          "pid": "1001",
          "firmware": [
            {
              "version": "1.2.3",
              "lastUpdated": "2025-10-01T12:00:00Z",
              "securityVersionNumber": 5,
              "minimumSecurityVersionNumber": 3,
              "imageRole": "Active"
            }
          ]
        },
```

```
      {
        "componentClass": "processor",
        "componentId": "CPU-002",
        "manufacturerSerialNumber": "CPU-SN-22222",
        "vid": "8086",
        "pid": "1002",
        "firmware": [
          {
            "version": "1.2.3",
            "lastUpdated": "2025-10-01T12:00:00Z",
            "securityVersionNumber": 5,
            "minimumSecurityVersionNumber": 3,
            "imageRole": "Active"
          }
        ]
      },
      {
        "componentClass": "memory",
        "componentId": "MEM-001",
        "manufacturerSerialNumber": "MEM-SN-33333",
        "vid": "1AD9",
        "pid": "3001",
        "firmware": [
          {
            "version": "N/A"
          }
        ]
      },
      {
        "componentClass": "memory",
        "componentId": "MEM-002",
        "manufacturerSerialNumber": "MEM-SN-44444",
        "vid": "1AD9",
        "pid": "3002",
        "firmware": [
          {
            "version": "N/A"
          }
        ]
      },
      {
        "componentClass": "bmc",
        "componentId": "BMC-001",
        "manufacturerSerialNumber": "BMC-SN-55555",
        "vid": "1A03",
        "pid": "2600",
        "firmware": [
          {
            "version": "3.4.5",
            "lastUpdated": "2025-09-15T08:00:00Z",
            "securityVersionNumber": 2,
            "minimumSecurityVersionNumber": 1,
            "imageRole": "Active"
          }
        ]
      }
    ],
    "subAssemblies": [
      {
        "assemblyName": "HotSwap Backplane",
        "assemblyId": "HSBP-001",
        "manufacturerSerialNumber": "HSBP-SN-66666",
        "vid": "0CC0",
        "pid": "0003",
        "components": [
          {
            "componentClass": "storageDevice",
            "componentId": "NVME-001",
```

```
        "manufacturerSerialNumber": "NVME-SN-77777",
        "vid": "144D",
        "pid": "7001",
        "firmware": [
          {
            "version": "2.1.0"
          }
        ]
      },
      {
        "componentClass": "storageDevice",
        "componentId": "NVME-002",
        "manufacturerSerialNumber": "NVME-SN-88888",
        "vid": "144D",
        "pid": "7002",
        "firmware": [
          {
            "version": "2.1.0"
          }
        ]
      },
      {
        "componentClass": "networkInterface",
        "componentId": "NIC-001",
        "manufacturerSerialNumber": "NIC-SN-99999",
        "vid": "8086",
        "pid": "8001",
        "firmware": [
          {
            "version": "1.0.0"
          }
        ]
      },
      {
        "componentClass": "networkInterface",
        "componentId": "NIC-002",
        "manufacturerSerialNumber": "NIC-SN-10101",
        "vid": "8086",
        "pid": "8002",
        "firmware": [
          {
            "version": "1.0.0"
          }
        ]
      }
    ]
  },
  {
    "assemblyName": "UBB",
    "assemblyId": "UBB-001",
    "manufacturerSerialNumber": "UBB-SN-20202",
    "vid": "0CC0",
    "pid": "0004",
    "components": [
      {
        "componentClass": "managementController",
        "componentId": "ubb-bmc",
        "vid": "1A03",
        "pid": "2601",
        "firmware": [
          {
            "version": "4.2.0",
            "imageRole": "Active"
          }
        ]
      }
    ],
    "subAssemblies": [
```

```
      {
        "assemblyName": "OAM-1",
        "assemblyId": "OAM-001",
        "manufacturerSerialNumber": "OAM-SN-30303",
        "vid": "0CC0",
        "pid": "0101",
        "components": [
          {
            "componentClass": "accelerator",
            "componentId": "GPU-001",
            "manufacturerSerialNumber": "GPU-SN-40404",
            "vid": "8086",
            "pid": "9001",
            "firmware": [
              {
                "version": "510.85"
              }
            ]
          },
          {
            "componentClass": "managementController",
            "componentId": "AMC-001",
            "manufacturerSerialNumber": "AMC-SN-50505",
            "vid": "1050",
            "pid": "A001",
            "firmware": [
              {
                "version": "1.0.0"
              }
            ]
          }
        ]
      },
      {
        "assemblyName": "OAM-2",
        "assemblyId": "OAM-002",
        "manufacturerSerialNumber": "OAM-SN-60606",
        "vid": "0CC0",
        "pid": "0102",
        "components": [
          {
            "componentClass": "accelerator",
            "componentId": "GPU-002",
            "manufacturerSerialNumber": "GPU-SN-70707",
            "vid": "8086",
            "pid": "9002",
            "firmware": [
              {
                "version": "510.85"
              }
            ]
          },
          {
            "componentClass": "managementController",
            "componentId": "AMC-002",
            "manufacturerSerialNumber": "AMC-SN-80808",
            "vid": "1050",
            "pid": "A002",
            "firmware": [
              {
                "version": "1.0.0"
              }
            ]
          }
        ]
      }
    ]
  }
```

```
        ]
      }
    ]
  }
```

Appendix B: Status Dump Examples

Below is a JSON example for Status Dump of a hypothetical node with:

- Dual-socket CPUs
- Memory modules
- BMC
- Two NVMe drives
- Two NICs on a hot-swap backplane
- UBB subassembly with BMC
- Two OAM subassemblies (each with GPU and AMC)


```
{
  "nodeId": "node-001",
  "nodeType": "server",
  "hostName": "ai-server-01",
  "vid": "0CC0",
  "pid": "0001",
  "healthy": false,
  "mode": "Awake",
  "status": "Crash",
  "lastError": "1D10T",
  "errorLog": [
    "1D10T: System Halt",
    "PEBKAC: OverVoltage"
  ],
  "lifecycle": "production",
  "power": {
    "power": 22000,
    "powerUnit": "Watt",
    "voltage": 220,
    "voltageUnit": "Volt",
    "current": 100,
    "currentUnit": "Amps"
  },
  "temperature": {
    "temperature": 125,
    "temperatureUnit": "Degrees C"
  },
  "physicalLocation": {
    "rack": "R12",
    "row": "Row-A",
    "datacenter": "DC1",
    "placement": {
      "type": "rackUnit",
      "slot": 20,
      "heightInU": 4
    }
  },
  "assemblies": [
    {
      "assemblyName": "MainBoard",
      "assemblyId": "mb-001",
      "vid": "0CC0",
      "pid": "0002",
      "healthy": false,
      "mode": "Awake",
      "status": "Crash",
      "lastError": "1D10T",
      "errorLog": [
        "1D10T: System Halt",
        "PEBKAC: OverVoltage",
        "501: Server Error"
```

```
  ],
  "lifecycle": "production",
  "power": {
    "power": 3000,
    "powerUnit": "Watt",
    "voltage": 60,
    "voltageUnit": "Volt",
    "current": 50,
    "currentUnit": "Amps"
  },
  "temperature": {
    "temperature": 125,
    "temperatureUnit": "Degrees C"
  },
  "components": [
    {
      "componentClass": "processor",
      "componentId": "cpu-0",
      "vid": "8086",
      "pid": "1000",
      "physicalLocation": {
        "slot": "CPU0",
        "position": "Socket-0"
      },
      "healthy": false,
      "mode": "Awake",
      "status": "Crash",
      "lastError": "1D10T",
      "errorLog": [
        "1D10T: System Halt",
        "PEBKAC: OverVoltage",
        "501: Server Error"
      ],
      "lifecycle": "production",
      "power": {
        "power": 165,
        "powerUnit": "Watt",
        "voltage": 3.3,
        "voltageUnit": "Volt",
        "current": 50,
        "currentUnit": "Amps"
      },
      "temperature": {
        "temperature": 125,
        "temperatureUnit": "Degrees C"
      },
      "utilization": {
        "CPU": 100,
        "PCIe": 80,
        "UPI": 10
      },
      "interface": "Northbridge",
      "communication": "JEDEC DDR",
      "dataRate": 6400,
      "dataRateValue": "MT/s",
      "performance": {
        "Packet Loss/Resend": 13,
        "Thermal Throttling": 15,
        "ECC": 0
      },
      "latency": 0,
      "latencyUnit": "us",
      "errorRate": 0,
      "errorUnit": "errors per second"
    },
    {
      "componentClass": "processor",
      "componentId": "cpu-1",
```

```
    "vid": "8086",
    "pid": "1001",
    "physicalLocation": {
      "slot": "CPU1",
      "position": "Socket-1"
    },
    "healthy": false,
    "mode": "Awake",
    "status": "Crash",
    "lastError": "1D10T",
    "errorLog": [
      "1D10T: System Halt"
    ],
    "lifecycle": "production",
    "power": {
      "power": 165,
      "powerUnit": "Watt",
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 50,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 75,
      "temperatureUnit": "Degrees C"
    },
    "utilization": {
      "CPU": 30,
      "PCIe": 10,
      "UPI": 10
    },
    "interface": "Northbridge",
    "communication": "JEDEC DDR",
    "dataRate": 6400,
    "dataRateValue": "MT/s",
    "performance": {
      "Packet Loss/Resend": 13,
      "Thermal Throttling": 15,
      "ECC": 0
    },
    "latency": 0,
    "latencyUnit": "us",
    "errorRate": 0,
    "errorUnit": "errors per second"
  },
  {
    "componentClass": "memory",
    "componentId": "dimm-0",
    "vid": "1AD9",
    "pid": "6405",
    "physicalLocation": {
      "slot": "DIMM-A1",
      "position": "Channel-A"
    },
    "healthy": false,
    "mode": "Awake",
    "status": "Crash",
    "lastError": "UCB10",
    "errorLog": [
      "UCB10: Uncorrectable Error Bank 10",
      "UCB09: Uncorrectable Error Bank 9",
      "UCB08: Uncorrectable Error Bank 8",
      "UCB07: Uncorrectable Error Bank 7"
    ],
    "lifecycle": "production",
    "power": {
      "power": 6.6,
      "powerUnit": "Watt",
```

```
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 2,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 75,
      "temperatureUnit": "Degrees C"
    }
  },
  {
    "componentClass": "memory",
    "componentId": "dimm-1",
    "vid": "1AD9",
    "pid": "6405",
    "physicalLocation": {
      "slot": "DIMM-B1",
      "position": "Channel-B"
    },
    "healthy": true,
    "mode": "Awake",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 6.6,
      "powerUnit": "Watt",
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 2,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 75,
      "temperatureUnit": "Degrees C"
    }
  },
  {
    "componentClass": "bmc",
    "componentId": "bmc-0",
    "vid": "1A03",
    "pid": "2600",
    "physicalLocation": {
      "slot": "BMC-0",
      "position": "MainBoard"
    },
    "healthy": true,
    "mode": "Awake",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 3.3,
      "powerUnit": "Watt",
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 1,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 52,
      "temperatureUnit": "Degrees C"
    },
    "utilization": {
      "CPU": 15,
      "Memory": 40,
```

```
          "Network": 10
        },
        "interface": "RMII",
        "communication": "NC-SI",
        "dataRate": 1000,
        "dataRateValue": "Mbps",
        "latency": 1,
        "latencyUnit": "us",
        "errorRate": 0,
        "errorUnit": "errors per second"
      }
    ]
  },
  {
    "assemblyName": "HotSwapBackplane",
    "assemblyId": "bp-nvme-01",
    "vid": "0CC0",
    "pid": "0003",
    "healthy": true,
    "mode": "Awake",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 60,
      "powerUnit": "Watt",
      "voltage": 12,
      "voltageUnit": "Volt",
      "current": 5,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 85,
      "temperatureUnit": "Degrees C"
    },
    "components": [
      {
        "componentClass": "storageDevice",
        "componentId": "nvme-0",
        "vid": "144D",
        "pid": "A384",
        "physicalLocation": {
          "slot": "NVMe-1",
          "position": "Front"
        },
        "healthy": true,
        "mode": "Active",
        "status": "Healthy",
        "lastError": "NULL",
        "errorLog": [],
        "lifecycle": "production",
        "power": {
          "power": 9.6,
          "powerUnit": "Watt",
          "voltage": 12,
          "voltageUnit": "Volt",
          "current": 0.8,
          "currentUnit": "Amps"
        },
        "temperature": {
          "temperature": 45,
          "temperatureUnit": "Degrees C"
        },
        "utilization": {
          "CapacityUsed": 72,
          "ReadBandwidth": 40,
          "WriteBandwidth": 15
```

```
    },
    "interface": "PCIe Gen4 x4",
    "communication": "NVMe",
    "dataRate": 64,
    "dataRateValue": "GT/s",
    "latency": 80,
    "latencyUnit": "us",
    "errorRate": 0,
    "errorUnit": "errors per second"
  },
  {
    "componentClass": "storageDevice",
    "componentId": "nvme-1",
    "vid": "144D",
    "pid": "A385",
    "physicalLocation": {
      "slot": "NVMe-2",
      "position": "Front"
    },
    "healthy": true,
    "mode": "Active",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 8.4,
      "powerUnit": "Watt",
      "voltage": 12,
      "voltageUnit": "Volt",
      "current": 0.7,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 43,
      "temperatureUnit": "Degrees C"
    },
    "utilization": {
      "CapacityUsed": 65,
      "ReadBandwidth": 20,
      "WriteBandwidth": 10
    },
    "interface": "PCIe Gen4 x4",
    "communication": "NVMe",
    "dataRate": 64,
    "dataRateValue": "GT/s",
    "latency": 75,
    "latencyUnit": "us",
    "errorRate": 0,
    "errorUnit": "errors per second"
  },
  {
    "componentClass": "networkInterface",
    "componentId": "nic-0",
    "vid": "8086",
    "pid": "1592",
    "physicalLocation": {
      "slot": "NIC-1",
      "position": "HotSwap-Bay-1"
    },
    "healthy": true,
    "mode": "Active",
    "status": "LinkUp",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 18,
```

```
        "powerUnit": "Watt",
        "voltage": 12,
        "voltageUnit": "Volt",
        "current": 1.5,
        "currentUnit": "Amps"
      },
      "temperature": {
        "temperature": 58,
        "temperatureUnit": "Degrees C"
      },
      "utilization": {
        "TxBandwidth": 40,
        "RxBandwidth": 35,
        "LinkUtilization": 38
      },
      "interface": "PCIe Gen5 x16",
      "communication": "Ethernet",
      "dataRate": 100,
      "dataRateValue": "Gbps",
      "latency": 2,
      "latencyUnit": "us",
      "errorRate": 0,
      "errorUnit": "errors per second"
    },
    {
      "componentClass": "networkInterface",
      "componentId": "nic-1",
      "vid": "8086",
      "pid": "1593",
      "physicalLocation": {
        "slot": "NIC-2",
        "position": "HotSwap-Bay-2"
      },
      "healthy": true,
      "mode": "Active",
      "status": "LinkUp",
      "lastError": "NULL",
      "errorLog": [],
      "lifecycle": "production",
      "power": {
        "power": 15.6,
        "powerUnit": "Watt",
        "voltage": 12,
        "voltageUnit": "Volt",
        "current": 1.3,
        "currentUnit": "Amps"
      },
      "temperature": {
        "temperature": 56,
        "temperatureUnit": "Degrees C"
      },
      "utilization": {
        "TxBandwidth": 15,
        "RxBandwidth": 12,
        "LinkUtilization": 14
      },
      "interface": "PCIe Gen5 x16",
      "communication": "Ethernet",
      "dataRate": 100,
      "dataRateValue": "Gbps",
      "latency": 2,
      "latencyUnit": "us",
      "errorRate": 0,
      "errorUnit": "errors per second"
    }
  ]
},
{
```

```
"assemblyName": "UBB",
"assemblyId": "ubb-01",
"vid": "0CC0",
"pid": "0004",
"healthy": true,
"mode": "Awake",
"status": "Healthy",
"lastError": "NULL",
"errorLog": [],
"lifecycle": "production",
"power": {
  "power": 3000,
  "powerUnit": "Watt",
  "voltage": 60,
  "voltageUnit": "Volt",
  "current": 50,
  "currentUnit": "Amps"
},
"temperature": {
  "temperature": 85,
  "temperatureUnit": "Degrees C"
},
"components": [
  {
    "componentClass": "managementController",
    "componentId": "ubb-bmc",
    "vid": "1A03",
    "pid": "2600",
    "physicalLocation": {
      "slot": "UBB-BMC-0",
      "position": "UBB"
    },
    "healthy": true,
    "mode": "Awake",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 3.3,
      "powerUnit": "Watt",
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 1,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 48,
      "temperatureUnit": "Degrees C"
    },
    "utilization": {
      "CPU": 12,
      "Memory": 35,
      "Network": 8
    },
    "interface": "USB",
    "communication": "redfish",
    "dataRate": 480,
    "dataRateValue": "Mbps",
    "latency": 1,
    "latencyUnit": "us",
    "errorRate": 0,
    "errorUnit": "errors per second"
  }
],
"subAssemblies": [
  {
    "assemblyName": "OAM-0",
```

```
"assemblyId": "oam-0",
"vid": "0CC0",
"pid": "0100",
"healthy": true,
"components": [
  {
    "componentClass": "accelerator",
    "componentId": "gpu-0",
    "vid": "8086",
    "pid": "1020",
    "physicalLocation": {
      "slot": "OAM-0",
      "position": "UBB-Slot-0"
    },
    "healthy": true,
    "mode": "Active",
    "status": "Healthy",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 960,
      "powerUnit": "Watt",
      "voltage": 48,
      "voltageUnit": "Volt",
      "current": 20,
      "currentUnit": "Amps"
    },
    "temperature": {
      "temperature": 72,
      "temperatureUnit": "Degrees C"
    },
    "utilization": {
      "Compute": 85,
      "Memory": 78,
      "Fabric": 65
    },
    "interface": "PCIe Gen5 x16",
    "communication": "PCIe",
    "dataRate": 800,
    "dataRateValue": "Gbps",
    "latency": 1,
    "latencyUnit": "us",
    "errorRate": 0,
    "errorUnit": "errors per second"
  },
  {
    "componentClass": "managementController",
    "componentId": "amc-0",
    "vid": "1050",
    "pid": "1000",
    "physicalLocation": {
      "slot": "AMC-0",
      "position": "OAM-0"
    },
    "healthy": true,
    "mode": "Awake",
    "status": "Enabled",
    "lastError": "NULL",
    "errorLog": [],
    "lifecycle": "production",
    "power": {
      "power": 1.65,
      "powerUnit": "Watt",
      "voltage": 3.3,
      "voltageUnit": "Volt",
      "current": 0.5,
      "currentUnit": "Amps"
```

```
      },
      "temperature": {
        "temperature": 45,
        "temperatureUnit": "Degrees C"
      },
      "utilization": {
        "CPU": 10,
        "Memory": 25,
        "FabricManagement": 20
      },
      "interface": "I2C",
      "communication": "PLDM over MCTP",
      "dataRate": 1,
      "dataRateValue": "Mbps",
      "latency": 5,
      "latencyUnit": "ms",
      "errorRate": 0,
      "errorUnit": "errors per second"
    }
  ]
},
{
  "assemblyName": "OAM-1",
  "assemblyId": "oam-1",
  "vid": "0CC0",
  "pid": "0101",
  "healthy": true,
  "components": [
    {
      "componentClass": "accelerator",
      "componentId": "gpu-1",
      "vid": "8086",
      "pid": "1021",
      "physicalLocation": {
        "slot": "OAM-1",
        "position": "UBB-Slot-1"
      },
      "healthy": true,
      "mode": "Active",
      "status": "Enabled",
      "lastError": "NULL",
      "errorLog": [],
      "lifecycle": "production",
      "power": {
        "power": 960,
        "powerUnit": "Watt",
        "voltage": 48,
        "voltageUnit": "Volt",
        "current": 20,
        "currentUnit": "Amps"
      },
      "temperature": {
        "temperature": 69,
        "temperatureUnit": "Degrees C"
      },
      "utilization": {
        "Compute": 62,
        "Memory": 55,
        "Fabric": 48
      },
      "interface": "PCIe Gen5 x16",
      "communication": "PCIe",
      "dataRate": 800,
      "dataRateValue": "Gbps",
      "latency": 1,
      "latencyUnit": "us",
      "errorRate": 0,
      "errorUnit": "errors per second"
```

```
            },
            {
              "componentClass": "managementController",
              "componentId": "amc-1",
              "vid": "1050",
              "pid": "1001",
              "physicalLocation": {
                "slot": "AMC-1",
                "position": "OAM-1"
              },
              "healthy": true,
              "mode": "Awake",
              "status": "Enabled",
              "lastError": "NULL",
              "errorLog": [],
              "lifecycle": "production",
              "power": {
                "power": 1.65,
                "powerUnit": "Watt",
                "voltage": 3.3,
                "voltageUnit": "Volt",
                "current": 0.5,
                "currentUnit": "Amps"
              },
              "temperature": {
                "temperature": 44,
                "temperatureUnit": "Degrees C"
              },
              "utilization": {
                "CPU": 9,
                "Memory": 22,
                "FabricManagement": 18
              },
              "interface": "I2C",
              "communication": "PLDM over MCTP",
              "dataRate": 1,
              "dataRateValue": "Mbps",
              "latency": 5,
              "latencyUnit": "ms",
              "errorRate": 0,
              "errorUnit": "errors per second"
            }
          ]
        }
      ]
    }
  ]
}
```

References

[1] "OCP Fault Management Infrastructure Requirements." Open Compute Project, Aug. 15, 2025. Available: https://www.opencompute.org/documents/ocp-fault-management-infrastructure-requirements-1-5-pdf

[2] "OCP GPU & Accelerator RAS Requirements." Open Compute Project, Oct. 23, 2025. Available: https://www.opencompute.org/documents/ocp-gpu-and-accelerators-ras-requirements-v1-7-10-23-2025-pdf

[3] "OCP RAS API Specification." Open Compute Project, Dec. 14, 2025. Available: https://www.opencompute.org/documents/2025-ocp-ras-api-v0-9-final-pdf

[4] "OCP Hyperscale CPU Debug and RAS Requirements." Open Compute Project, Sept. 15, 2025. Available: https://www.opencompute.org/documents/hyperscale-cpu-ras-and-debug-requirements-specification-v0-7-09-29-2025-pdf